

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ 컨벡스



□ 라그랑주

제약이 있는 경우에도 genereli 하게 구하고싶어서 .

□ 그래디언트 디센트

AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-1. 컨벡스 **셋**

Set (집합)

직선(line)과 선분(line segment)

두 점 x_1 과 x_2 를 잇는 선 사이에 있는 점 y 를 아래와 같이 표현

$$y = wx_1 + (1 - w)x_2$$



- 직선(line): 시작과 끝 지점이 존재하지 않음
- 선분(line segment): 시작과 끝 지점이 존재함

아핀 셋(affine set) 직선을 포함하는 집합

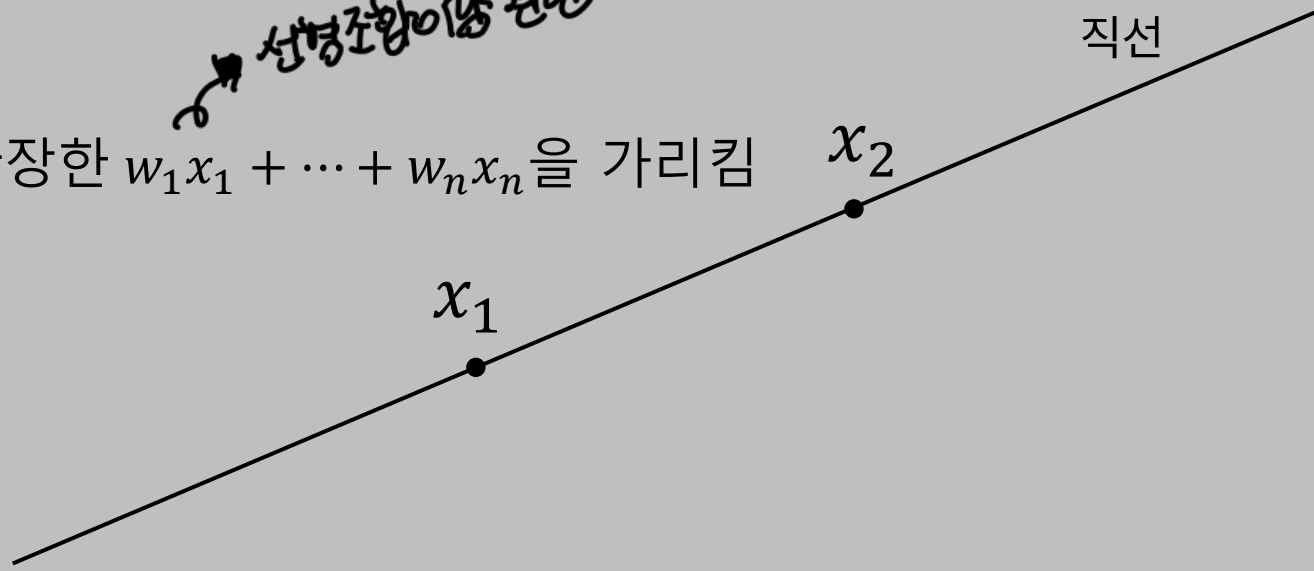
직선을 포함하는 집합(무한)

- $w x_1 + (1 - w) x_2 \in C$ 이면 집합 C 를 아핀 셋이라고 함.
- 두 점 x_1, x_2 를 잇는 직선은 집합 C 에 포함되어야 한다는 뜻
- 점은 w 값의 제한이 없으므로 직선에 해당

- 회색 영역이 아핀 셋을 의미

- 아핀 조합(affine combination): n 차원으로 확장한 $w_1 x_1 + \dots + w_n x_n$ 을 가리킴

선형조합이랑 관련



아핀 함수 vs 선형 함수

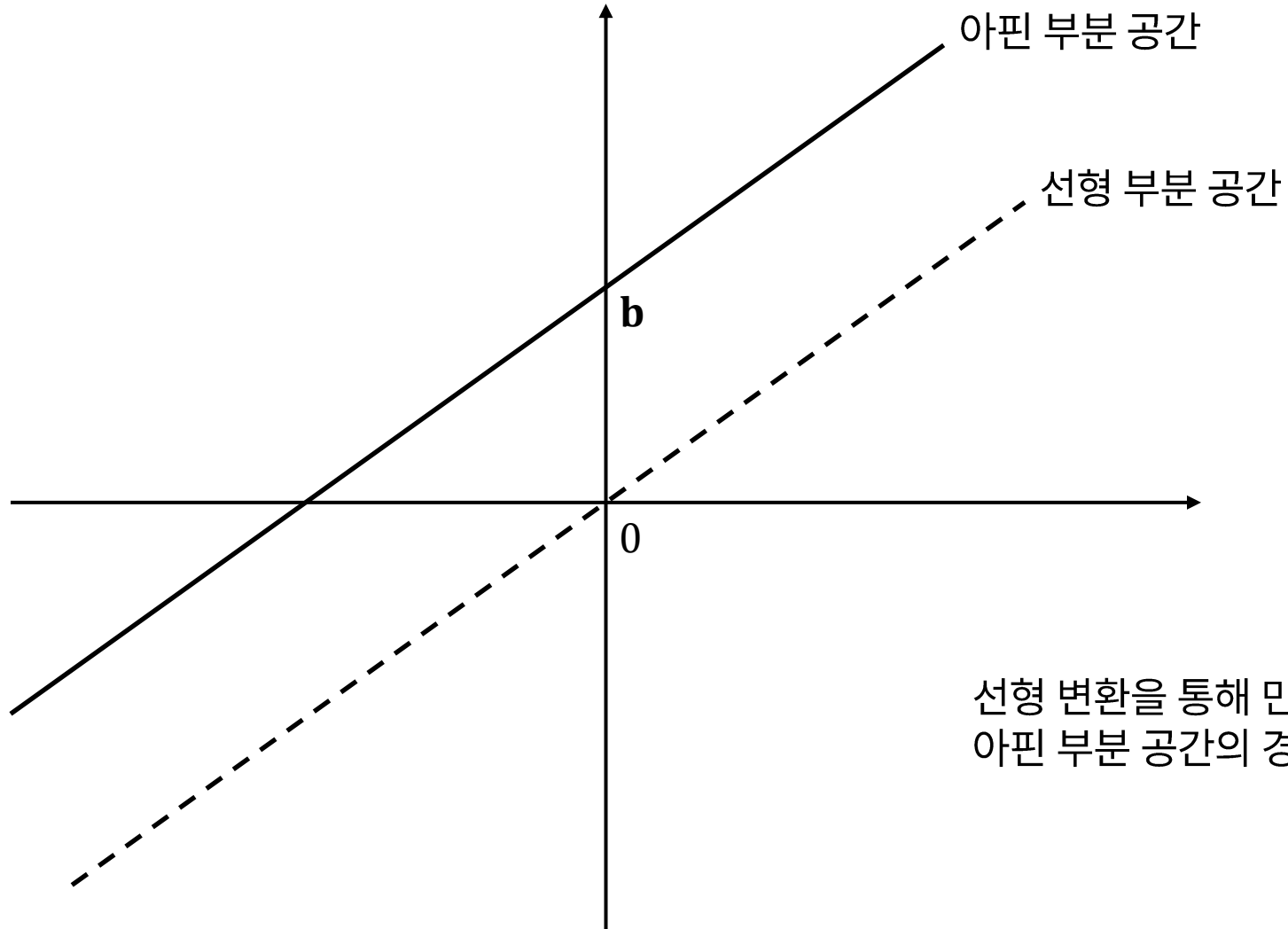
$$f: \mathbb{R}^m \rightarrow \mathbb{R}^n$$

함수 f 는 n 차원 공간에 속하는 벡터를 m 차원 공간에 속하는 벡터로 변환 한다는 뜻

linear function: $f(\mathbf{x}) = W\mathbf{x}$ ~~절편~~ X

affine function: $f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$ ~~절편~~ O

아핀 함수 vs 선형 함수



선형 변환을 통해 만들어진 선형 부분 공간은 원점을 지나는 반면,
아핀 부분 공간의 경우 상수 b 에 의해 y 절편이 존재함

컨벡스 셋(convex set)

- 만약 집합 C 내부의 두 점 사이의 선분이 집합 C 에 속한다면 집합 C 는 컨벡스(convex)합니다.
- 즉, 두 점 $x_1, x_2 \in C$ 에 대해 $0 \leq w \leq 1$ 을 만족하는 w 에 대해 아래 조건을 만족하면 집합 C 를 컨벡스 셋(convex set)라고 합니다.

$$wx_1 + (1 - w)x_2 \in C$$

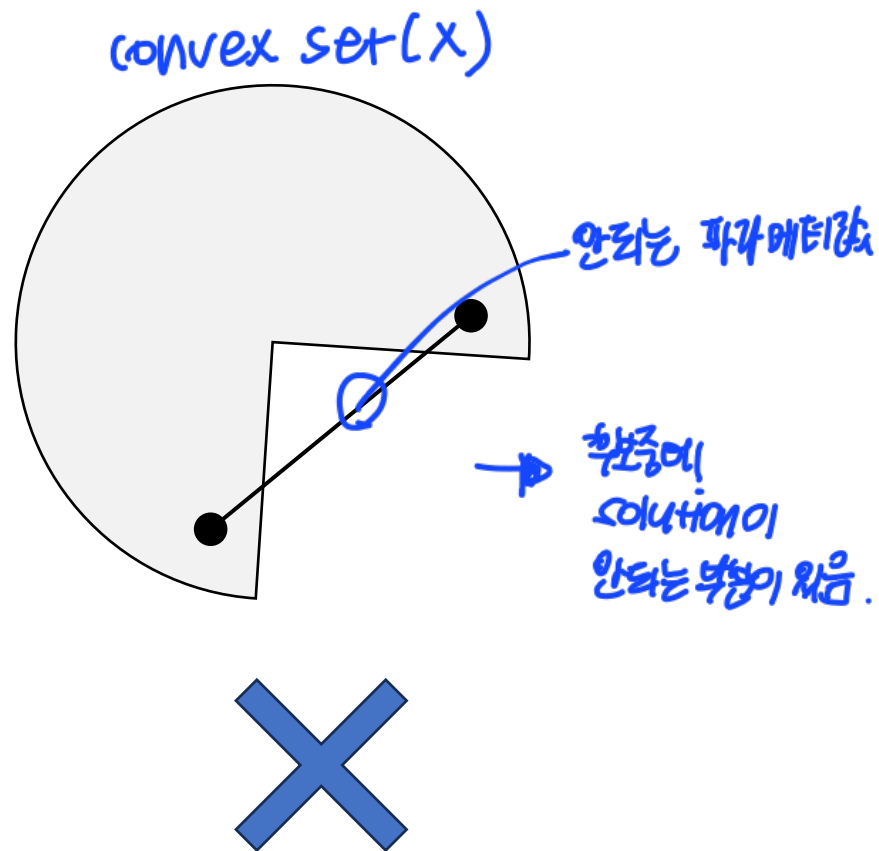
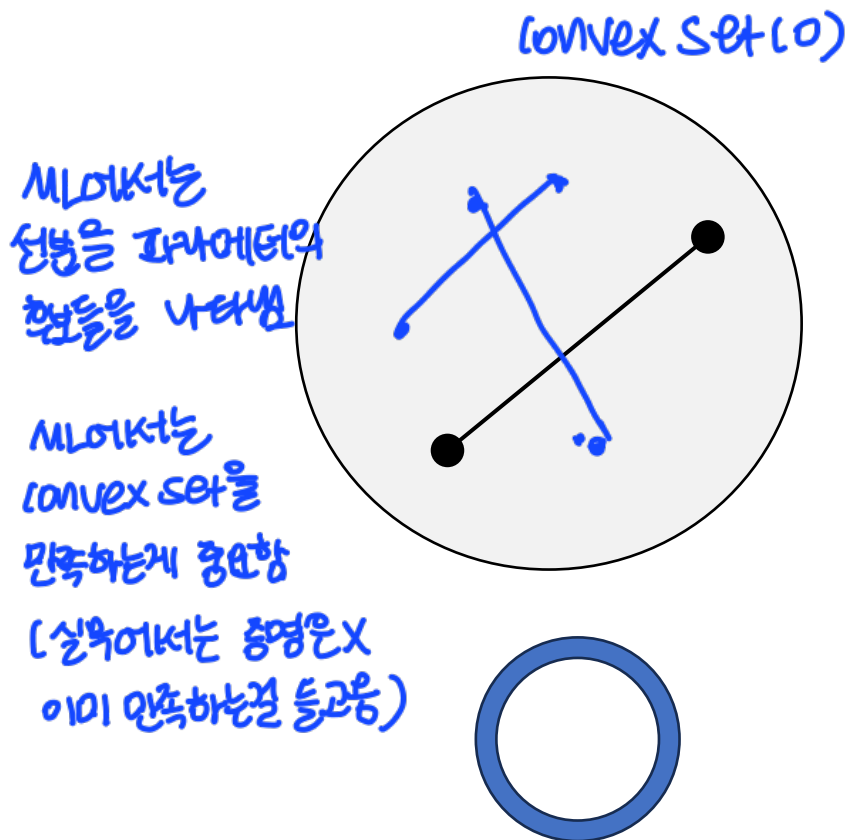
↓
선분. $0 \leq w \leq 1$ 이므로

컨벡스 셋(convex set) vs 아핀 셋(affine set) 차이

- 아핀 셋(affine set): 두 점을 잇는 직선(line)을 포함
- 컨벡스 셋(convex set): 두 점 사이의 선분(line segment)을 포함
- 아핀 셋(affine set): 가중치 w 값의 범위 제한이 없음
- 컨벡스 셋(convex set): 가중치 w 값의 범위 제한이 있음 ($0 \leq w \leq 1$) → *미리서는 이게 중요*

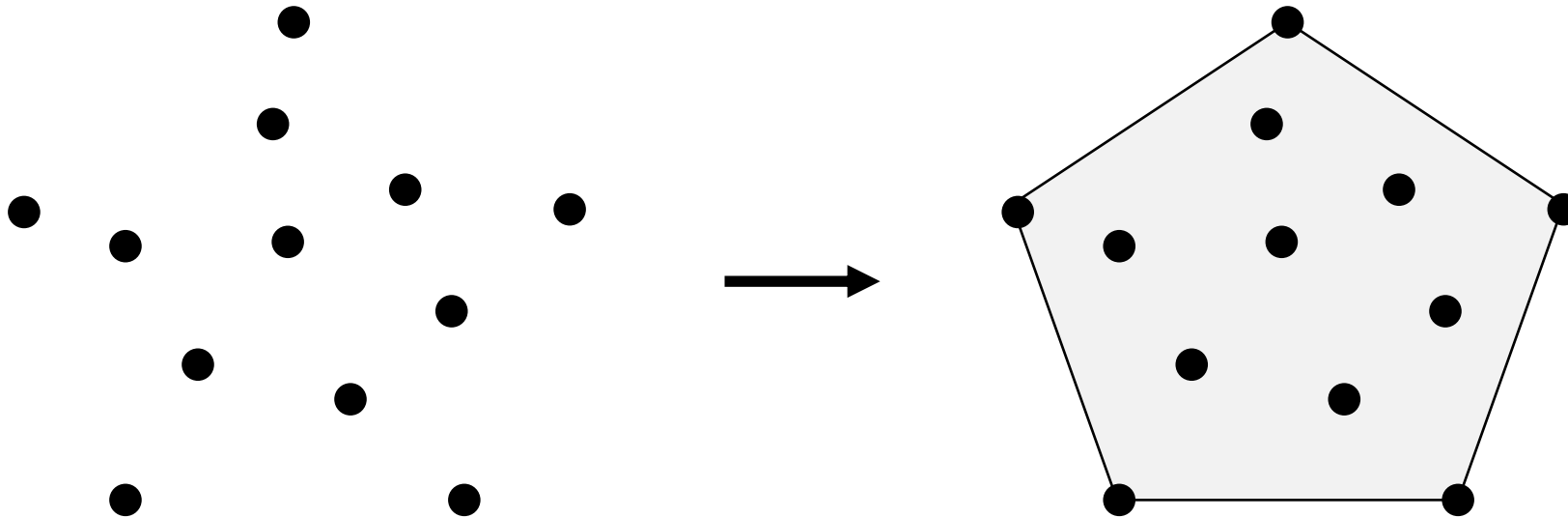
컨벡스셋 - 예시

컨벡스셋 내의 모든 두 점을 서로 이었을 때, 해당 선분이 집합 내에 속해야 함



컨벡스 헐(convex hull)

- 컨벡스 헐은 주어진 점들을 포함하는 컨벡스 셋을 의미



초평면(hyperplane)

집합기호

이걸 만족하는

내적값이 일정한 것.

$$\{\mathbf{x} \mid \mathbf{w}^T \mathbf{x} = b\}$$

가중치 집합

내재해서 결과가 b 인 집합.

- 초평면(hyperplane)이란 $\mathbf{w}^T \mathbf{x} = b$ 를 만족하는 데이터 포인트 $\mathbf{x}$ 의 집합을 의미
- $\mathbf{w}$ 는 n 차원 가중치 벡터, 즉 $\mathbf{w} \in \mathbb{R}^n$
- b 는 1차원 스칼라값, 즉, $b \in \mathbb{R}$
- $\mathbf{w}^T \mathbf{x} = b$ 는 벡터 $\mathbf{w}$ 와 벡터 $\mathbf{x}$ 를 내적인 값이 b 라는 것을 의미



$x_1 \cdot w \Rightarrow$ 초평면 위로 갔

$x_2 \cdot w \Rightarrow$ 초평면 위로 갔

초평면 (hyperplane) : 구분하는 선.

: 내적의 결과를 모두 모아놓은 곳

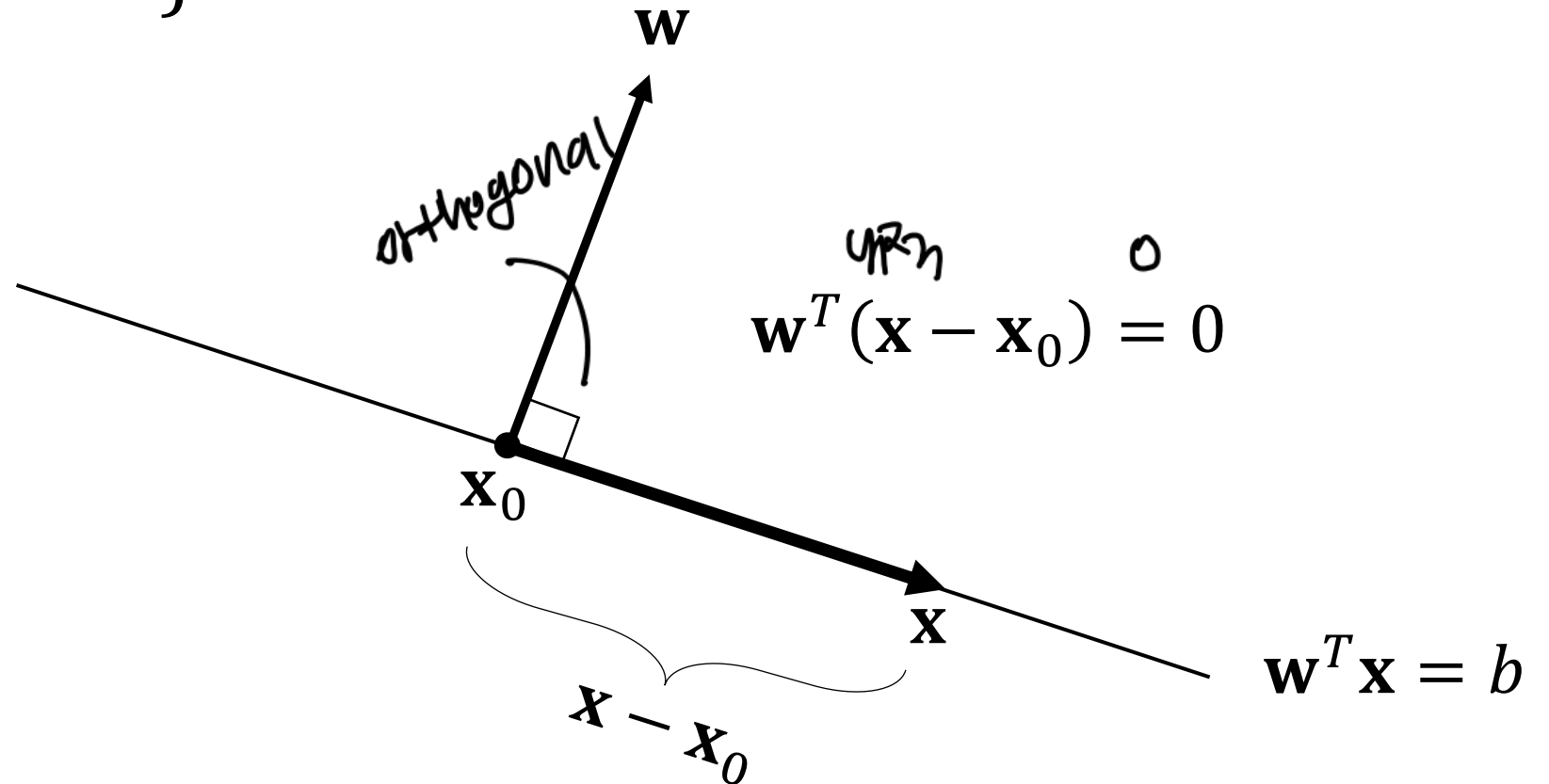
ML에서 분류하는건 초평면을 찾는것

- x 와 w 를 내적했는데 초평면보다 작으면 0
크면 Δ 로 분류하는 개념.

초평면(hyperplane)

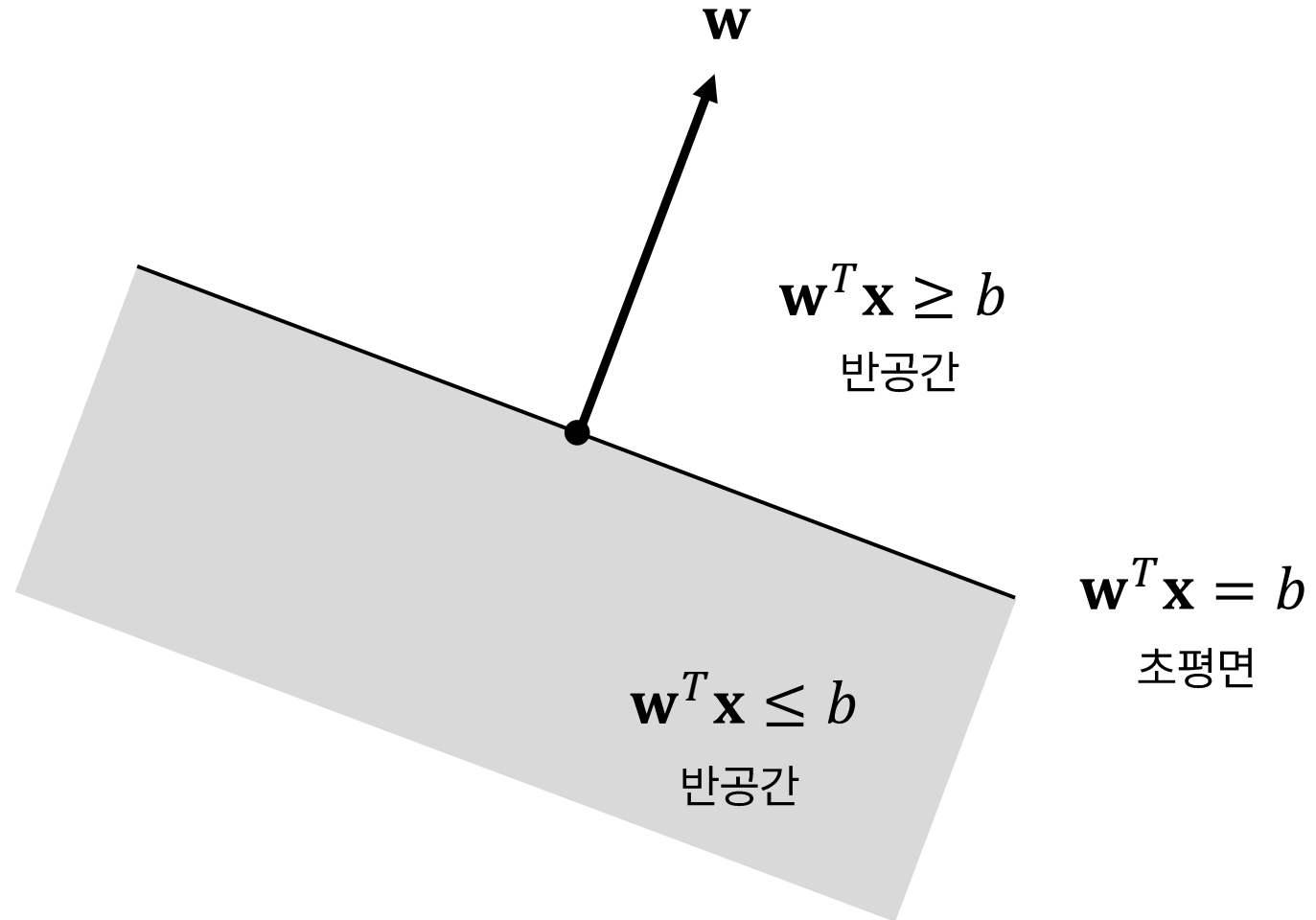
$$\{\mathbf{x} \mid \mathbf{w}^T (\mathbf{x} - \mathbf{x}_0) = 0\}$$

* $\mathbf{w}$ 를 구해야함. $\rightarrow$ 2차원 그림하듯 초평면이란
직선을 찾아짐

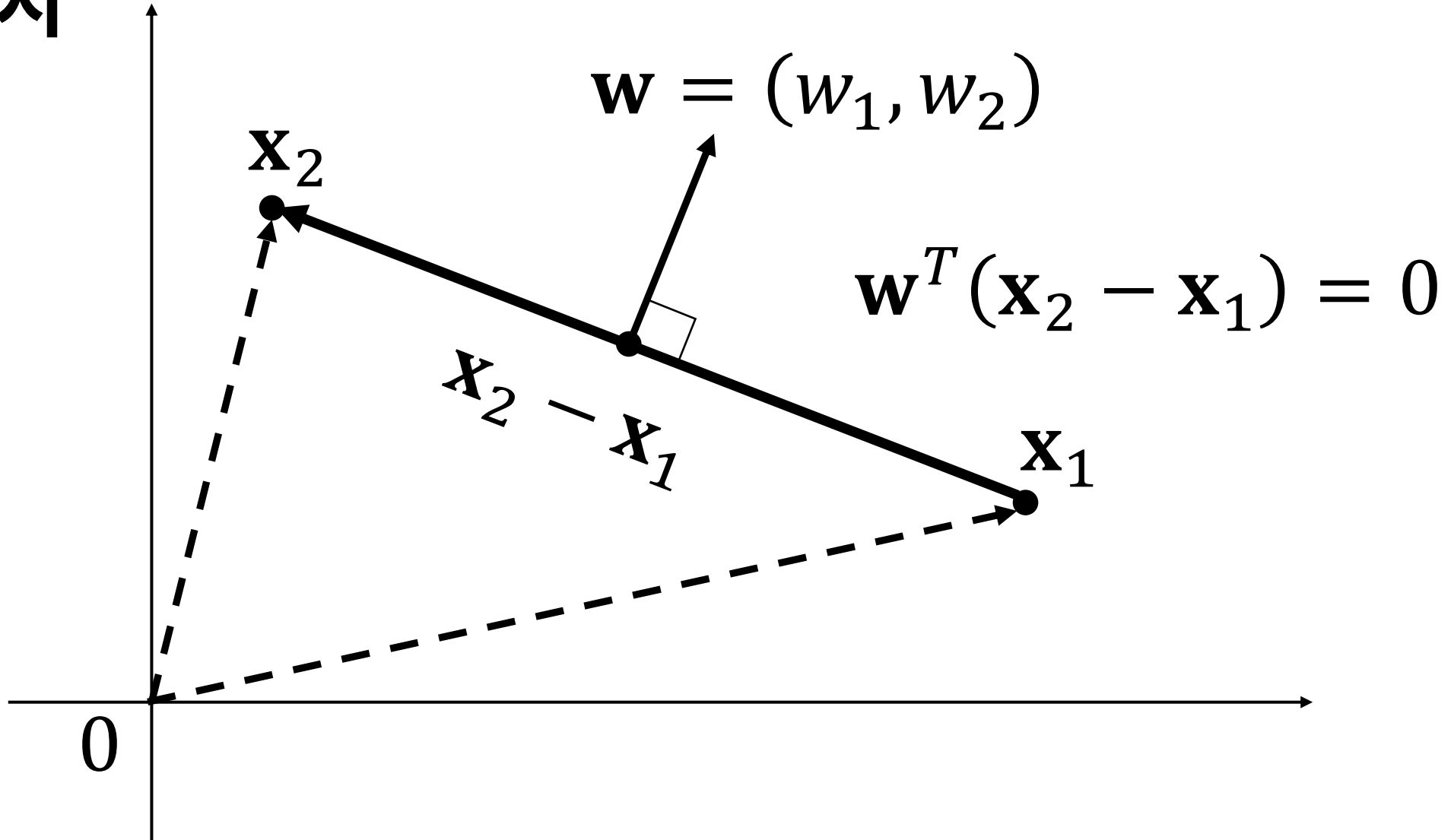


초평면과 반공간(halfspace)

- 전체 공간은 초평면에 의해 두 개의 반공간으로 나뉨
- 머신러닝의 목적은 전체 공간을 효과적으로 나눌 수 있는 초평면을 찾는 문제라고 할 수 있음



초평면 예시



AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-2. 컨벡스 함수

컨벡스 함수(convex function)

convex
아래로 볼록

concave
위로 볼록

$$f(w\mathbf{x}_1 + (1-w)\mathbf{x}_2) \leq wf(\mathbf{x}_1) + (1-w)f(\mathbf{x}_2)$$

를 만족하는 함수 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 를 컨벡스(convex)하다고 말함

두 점 $(\mathbf{x}_1, f(\mathbf{x}_1)), (\mathbf{x}_2, f(\mathbf{x}_2))$ 사이의 선분이 함수 f 의 그래프보다 위에 있어야 한다는 뜻

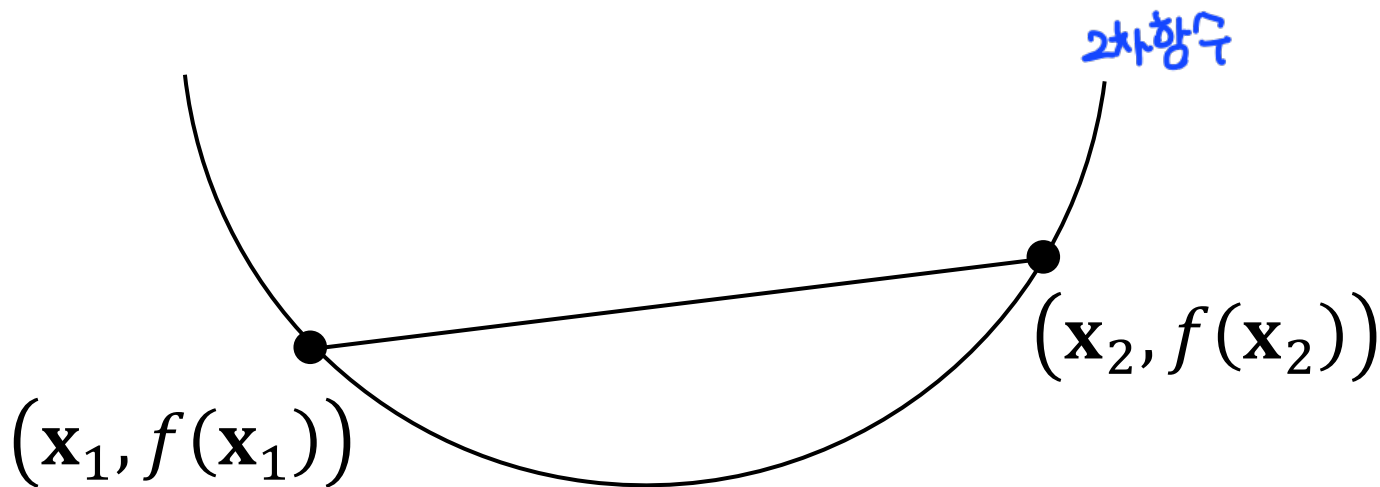
* 꼭짓점을 줄인다?

목적함수가 convex 함수여야함

: 정점을 해야함. but 어렵

→ 알려진 convex 함수를

같이 써 씀



1차 미분 조건

$$\nabla f(\mathbf{x}) = 0$$

feature의 영향력.

- ∇f 는 그래디언트(gradient)를 의미하며 모든 점에 대해 해당 점의 미분값을 의미함
- 그래디언트 값이 0일 때 최소값 성립

AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-3. 컨벡스 최적화

이름했으니까 최소화 찾기 위해 convex function으로 맞추는 것.

제약식(constraint)이 있는 최적화(optimization)

$$\begin{array}{ll} \min f(\mathbf{x}) \\ \text{subject to} \begin{cases} g_i(\mathbf{x}) \leq 0, & i = 1, \dots, m \\ h_i(\mathbf{x}) = 0, & i = 1, \dots, p \end{cases} \end{array}$$

~같은 제약조건 하에

$f(\mathbf{x})$: 목적함수(objective function)

$\mathbf{x}$: 최적화 변수

$g_i(\mathbf{x}) \leq 0$: 부등식 제약 함수(inequality constraint function)

$h_i(\mathbf{x}) = 0$: 등식 제약 함수(equality constraint function)

어떤 경우에는 제약조건이 없을 수도 있고 하나만 있을 수도 있고...

→ 시에서 다한 것

컨벡스 최적화(convex optimization)

$$\min f(\mathbf{x})$$

$$\text{subject to } \begin{cases} g_i(\mathbf{x}) \leq 0, & i = 1, \dots, m \\ \mathbf{w}_i^T \mathbf{x} = b_i, & i = 1, \dots, p \end{cases}$$

내적값이 일정해야 한다.

$f(\mathbf{x})$: 목적함수(objective function), 컨벡스 함수이어야 함 (반드시~)

→ 그 차항도 같은거 써라.

$\mathbf{x}$: 최적화 변수

$g_i(\mathbf{x}) \leq 0$: 부등식 제약 함수(inequality constraint function), 컨벡스 함수이어야 함

$h_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} - b_i$: 등식 제약 함수(equality constraint function), 아핀 함수이어야 함

컨벡스 최적화(convex optimization)

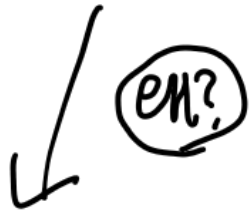
$$g_i(\mathbf{x}) \leq 0$$

왜 $g_i(\mathbf{x})$ 는 0보다 작아야 할까?

$g_i(\mathbf{x}) \leq 0$ 이어야 $g_i(\mathbf{x})$ 가 컨벡스 함

$g_i(\mathbf{x}) > 0$ 이면 $g_i(\mathbf{x})$ 는 컨벡스 하지 않음





예를 들어

목적 함수: $f(x) = x^2 - 1$

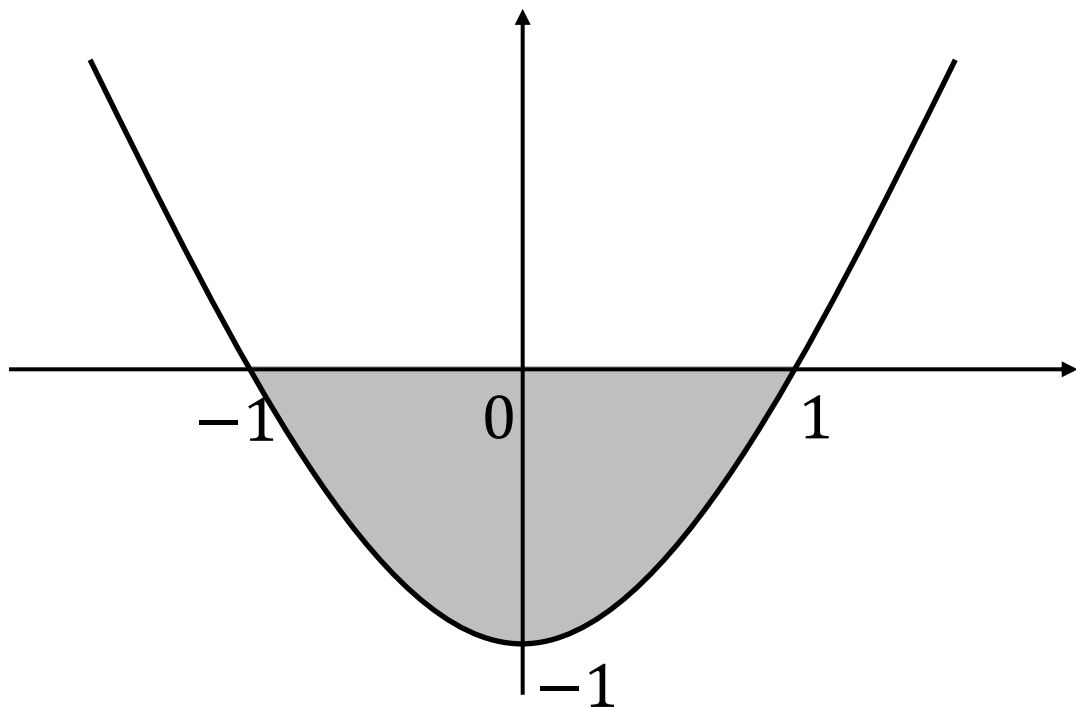
제약 함수: $f(x) \leq 0 \Leftrightarrow x^2 - 1 \leq 0$

제약 조건을 만족하는 변수 x

$$-1 \leq x \leq 1$$

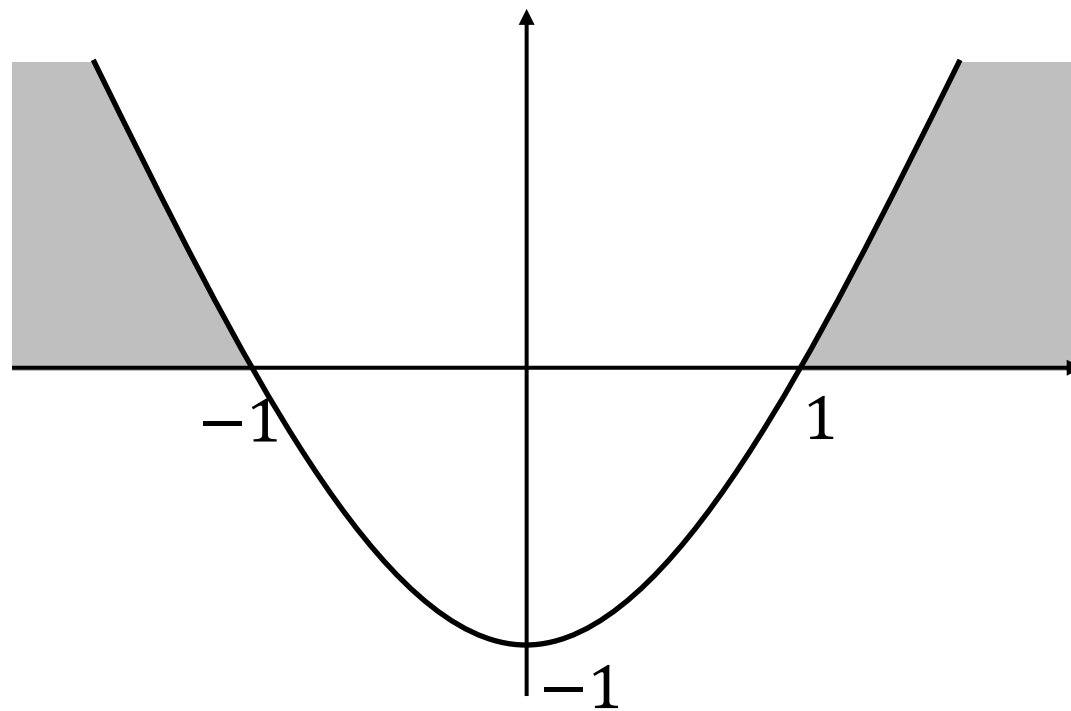
컨벡스 최적화(convex optimization)

$$f(x) = x^2 - 1 \leq 0$$



컨벡스 함

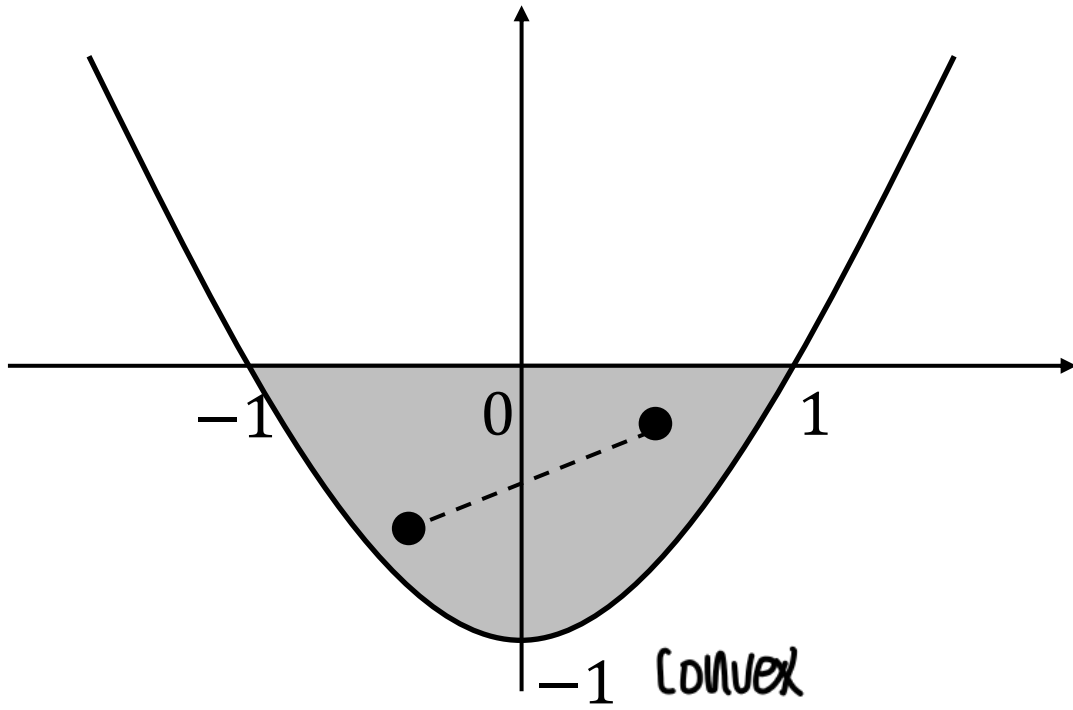
$$f(x) = x^2 - 1 \geq 0$$



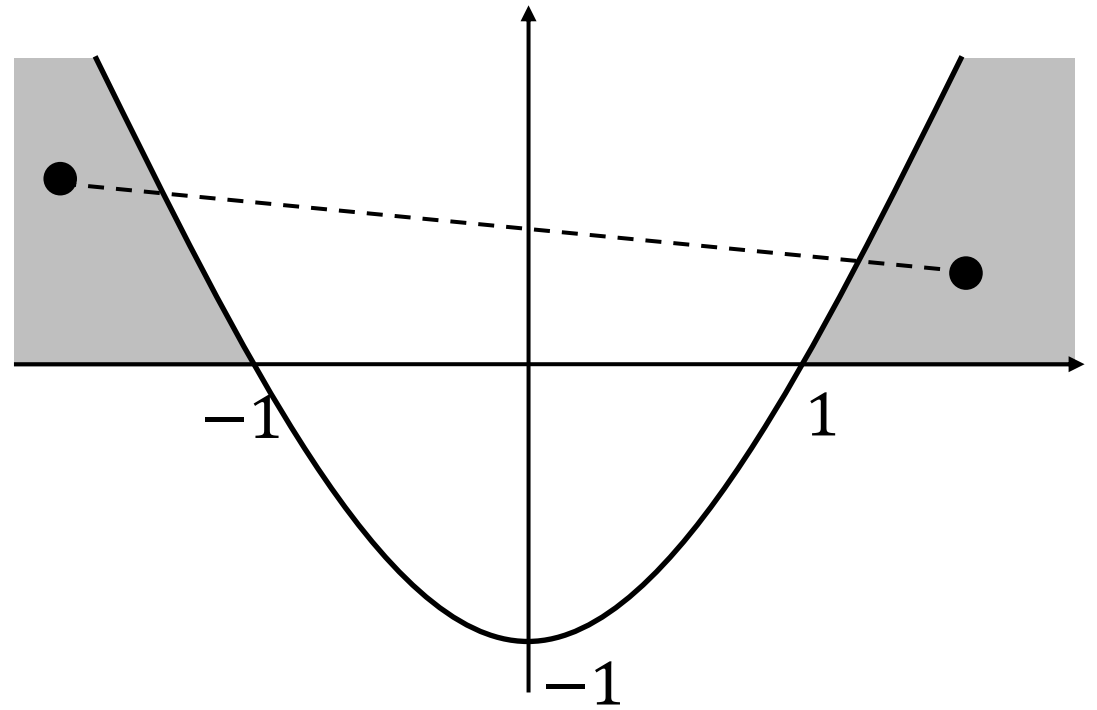
컨벡스 하지 않음

컨벡스 최적화(convex optimization)

$$f(x) = x^2 - 1 \leq 0$$



$$f(x) = x^2 - 1 \geq 0$$



AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-4. 라그랑주 프리멀 함수

(한개씩에 쓰고싶어서)

→ 목적함수의 제약조건 1줄로 작성하고 싶어서

부등식 제약조건

등식 제약조건

라그랑주 프리멀 함수(Lagrange primal function)

$$L_P(\mathbf{x}, \boldsymbol{\lambda}, \mathbf{v}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) + \sum_{i=1}^p v_i h_i(\mathbf{x})$$

· 제약조건 한번에 같이 표현

$f(\mathbf{x})$: 목적함수(objective function)

$\mathbf{x}$: 최적화 변수

$g_i(\mathbf{x}) \leq 0$: 부등식 제약 함수(inequality constraint function)

$h_i(\mathbf{x}) = 0$: 등식 제약 함수(equality constraint function)

λ_i : 제약식 $g_i(\mathbf{x}) \leq 0$ 에 대한 라그랑주 승수

v_i : 제약식 $h_i(\mathbf{x}) = 0$ 에 대한 라그랑주 승수

(ex)

$\lambda = 0$ 이거나 $\lambda = \infty$ 이라면
없앨 수 있음.

제약조건 강도를 결정함.

$\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_m), \mathbf{v} = (v_1, v_2, \dots, v_p)$: 듀얼 변수(dual variable) 또는 라그랑주 승수 벡터

라그랑주 프리멀 함수(Lagrange primal function)

예제)

$$\frac{\partial L_P}{\partial x_1} = 2x_1 + \lambda \equiv 0$$

$$L_P(x_1, x_2, \lambda) = f(x_1, x_2) + \lambda h(x_1, x_2) = x_1^2 + x_2^2 + \lambda(x_1 + x_2 - 2)$$

$$\leftrightarrow 2x_1 = -\lambda$$

위 문제를 풀기 위해 함수 $L(x_1, x_2)$ 를 x_1, x_2, λ 에 대해 편미분 결과 0이 되는 지점을 찾습니다.

$$\frac{\partial L}{\partial x_2} = 2x_2 + \lambda \equiv 0$$

$$\leftrightarrow 2x_2 = -\lambda$$

$$\therefore x_1 = 1, \quad x_2 = 1, \quad \lambda = -2$$

$$\frac{\partial L}{\partial \lambda} = x_1 + x_2 - 2 \equiv 0$$

$$\leftrightarrow x_1 + x_2 = 2$$

라그랑주 프리멀 함수(Lagrange primal function)

local minimum

라그랑주 프리멀 함수를 미분해서 0되는 지점을 찾다라고 해도
그 지점이 반드시 global 최적해라는 말은 아님.

→ 미분해서 0되는 지점이라도 global 최적해 아닐 수도 있음

→ 라그랑주 듀얼 함수 사용

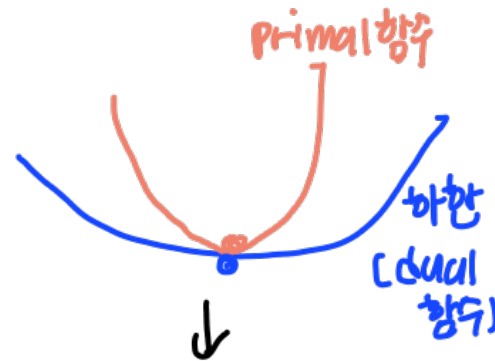
AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-5. 라그랑주 듀얼 함수

중요한 기대에 넣음.

라그랑주 듀얼 함수(Lagrange dual function)



$$L_D(\lambda, \mathbf{v}) = \inf_{\mathbf{x}} L(\mathbf{x}, \lambda, \mathbf{v}) = \inf_{\mathbf{x}} \left(f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) + \sum_{i=1}^p v_i h_i(\mathbf{x}) \right)$$

듀얼 함수에서는 $\mathbf{x}$ 를 최소화해서 식에 대입하므로 결과적으로는 $\mathbf{x}$ 가 남지 않고 $\lambda, \mathbf{v}$ 에 대한 함수가 됨

라그랑주 듀얼 함수: 라그랑주 프리멀 함수의 하한(lower bound)

라그랑주 듀얼 함수는 $\lambda \geq 0$ 에 대해 최적값 p^* 의 하한(lower bound)

$$\underbrace{L_D(\lambda, \mathbf{v})}_{\text{듀얼}} \leq \underbrace{p^*}_{\text{프리멀 optimal solution}}$$

$$\inf_{\mathbf{x}} f(\mathbf{x})$$

함수 $f(\mathbf{x})$ 의 하한을 구할 수 있는 $\mathbf{x}$ 를 찾아서 $\mathbf{x}$ 에 대한 $f(\mathbf{x})$ 값을 반환해라

라그랑주 듀얼 함수(Lagrange dual function)

라그랑주 듀얼 함수의 최적값을 d^* 라고 하면 아래와 같은 부등식이 성립합니다.

즉, 프리멀 문제의 최적값을 구하는 것은 듀얼 함수를 최대화하는 것과 동일합니다.

$$\underbrace{d^* \leq p^*}$$

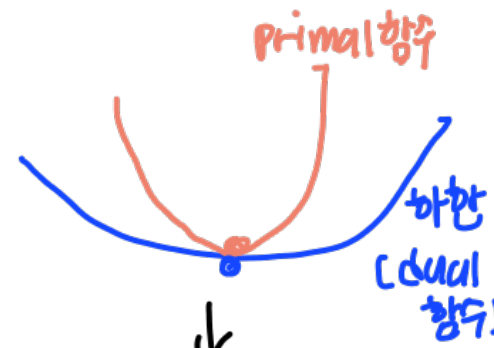
위와 같은 성질을 weak duality라고 합니다.

라그랑주 듀얼 함수의 최적값 d^* 는 라그랑주 프리멀 함수의 최적값 p^* 보다 작거나 같습니다.

이때 두 최적값의 차이 $p^* - d^*$ 를 듀얼리티 갭(duality gap)이라고 부릅니다.

듀얼리티 갭은 0보다 크거나 같습니다. 즉, nonnegative합니다.

↳ 이르면 참 죽겠다.



↓
차이가
같은 죽겠지만
다시하게 차이가 났.

:차이를 '듀얼리티 갭'
이라고 함

→ 갭이 0이면 좋을까?

AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-6. KKT(Karush-Kuhn-Tucker) 조건

↓
듀얼리티 값이 0인것을 알기위한 조건

KKT(Karush-Kuhn-Tucker) 조건

f_i, g_i 가 컨벡스 함수이고, h_i 가 아핀 함수일 때, 특정 지점 x^*, λ^*, v^* 에 대해 아래와 같은 KKT 조건을 만족한다면, $x^*, (\lambda^*, v^*)$ 는 프리멀 문제와 듀얼 문제에서의 최적값이며, 듀얼리티 갭이 0입니다.

이런 보충방식이 KKT조건.

KKT(Karush-Kuhn-Tucker) 조건

Primal feasibility 조건

→ 당연한거. 검증할 필요 X

$$\left. \begin{aligned} g_i(\mathbf{x}^*) &\leq 0, & i &= 1, \dots, m \\ h_i(\mathbf{x}^*) &= 0, & i &= 1, \dots, p \end{aligned} \right\}$$

$\mathbf{x}^*$ 가 프리멀 함수에서의 제약 조건을 만족한다는 것을 보여줌

Dual feasibility

→ 당연한대기

$$\lambda_i^* \geq 0, \quad i = 1, \dots, m$$

라그랑주 듀얼 함수를 생각하면 $\lambda_i^* \geq 0$ 을 만족한다는 사실을 알 수 있음

Complementary slackness

$$\lambda_i^* g_i(\mathbf{x}^*) = 0, \quad i = 1, \dots, m$$

다음 페이지에서 설명

Stationarity

$$\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(\mathbf{x}^*) + \sum_{i=1}^p v_i^* \nabla h_i(\mathbf{x}^*) = 0$$

→ 당연한대기

$\mathbf{x}$ 에 대한 그래디언트는 $\mathbf{x} = \mathbf{x}^*$ 에서 0이 됨

KKT(Karush-Kuhn-Tucker) 조건

Complementary slackness

$$\lambda_i^* g_i(\mathbf{x}^*) = 0, \quad i = 1, \dots, m$$

라그랑주 듀얼함수와 일치해야함

$$L_D(\lambda^*, \mathbf{v}^*) = \underline{L_P(\mathbf{x}^*, \lambda^*, \mathbf{v}^*)}$$

$$\begin{aligned} &= f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* g_i(\mathbf{x}^*) + \sum_{i=1}^p v_i^* h_i(\mathbf{x}^*) \\ &= f(\mathbf{x}^*) \end{aligned}$$

↓
미분하면 0됨

마지막 부분에서는 $h_i(\mathbf{x}^*) = 0$ 과 $\lambda_i^* g_i(\mathbf{x}^*) = 0$ 이라는 조건을 사용함

이는 프리멀 문제 최적값 $\mathbf{x}^*$ 와 듀얼 문제의 최적값 $(\lambda^*, \mathbf{v}^*)$ 는 제로 듀얼리티 갭이라는 것을 보여 주므로 프리멀 듀얼 최적값임을 알 수 있습니다.

정리하면, 컨벡스 최적화 문제에서 목적 함수와 제약 함수가 미분 가능할 때, KKT 조건을 만족하는 지점은 제로 듀얼리티 갭(zero duality gap)을 가지며 이는 프리멀 듀얼 최적값이라는 것을 의미

• 최적해 x^* 가 $g(x^*) < 0$ 의 영역에 있다면, 최적해 계산에 이 제약조건은 영향을 미치지 않음. $\lambda_i = 0$
• x^* 가 부등식 제약조건인 경계선인 $g(x^*) = 0$ 의 영역에 있다면, 등식 제약조건과 같고 하한형 제약이라고 함 } Complementary slackness 라고 함

KKT(Karush-Kuhn-Tucker) 조건

Complementary slackness

$$\lambda_i^* g_i(x^*) = 0, \quad i = 1, \dots, m$$

제약조건이 강하게 발생하는 상황(!?)

Ex) 노트북을 구매할 때 100만원 이하의 노트북을 선택해야 하는 상황

$$g(x) \leq 100 \quad g(x) \text{는 현재 가지고 있는 돈을 의미하고, 100만원을 초과하면 안됨}$$

(1) $g(x) = 100$ 인 경우, (내가 가진 돈이 100만원인 경우)

이 경우에는 제약조건이 중요하게 작용

→ 따라서 제약조건에 영향을 미치는 정도를 나타내는 변수인 λ 는 0보다 커질 수 있음

(2) $g(x) = 80$ 인 경우, (내가 가진 돈이 80만원인 경우)

이 경우에는 이미 제약조건을 충족함

→ 따라서 제약조건은 영향을 미치지 못함 (사실상 제약조건이 필요없음)

→ 따라서 제약조건에 영향을 미치는 정도를 나타내는 변수인 λ 는 0이 됨

AI/BigData 인텐시브 과정

Section 1. 최적화

Section 1-7. 머신러닝에서의 최적화



최소 제곱법

= loss의 제곱합 → 작을수록 좋다.

$$= \min \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

라그랑주 프네일 함수
이제 제약조건이 없는 경우

$$\min \sum_{i=1}^n (y_i - \underbrace{w^T x_i}_{= \hat{y}_i})^2$$

$$\begin{bmatrix} x_1 = \text{가격} & y = \text{노출된 판매량} \\ 2 & 3 \\ 1 & 2 \\ 5 & 4 \\ \vdots & \vdots \end{bmatrix} \quad w^T x = \hat{y} \text{ (추정량)} \quad \begin{bmatrix} 2 \\ 0 \\ 2 \\ \vdots \end{bmatrix}$$

두 배열 비교

$y_i - \hat{y}_i$: loss를 계산할 수 있다.

회귀 분석에서 구하려는 가중치 벡터 w 는 위 목적 함수를 최소화하는 값으로 정합니다.

위 식을 행렬 형태로 쓰면 아래와 같습니다.

목적 함수 $\Rightarrow \sum_{i=1}^n (y_i - \hat{y}_i)^2$

행렬

$$\min (y - Xw)^T (y - Xw)$$

Xw 해야 계산이 됨
(ex) 100×2

(=)

$$\Rightarrow \left[\begin{pmatrix} 3 \\ 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} \right]^T \left[\begin{pmatrix} 3 \\ 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} \right]$$

$$= (1 \ 2 \ 2) \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix} = 1 + 4 + 4 = 9$$

정답

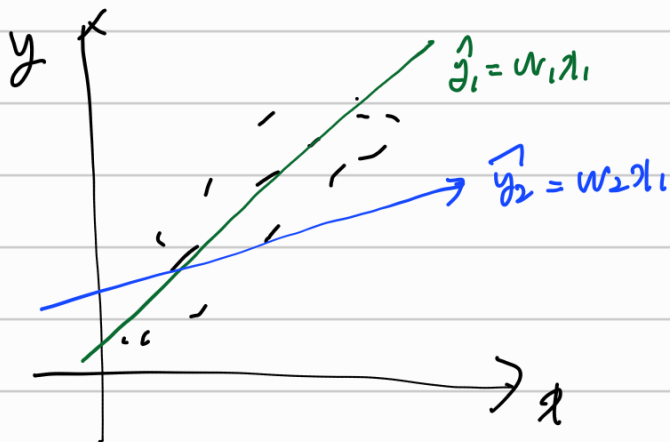
(제곱으로 되어있는데 자기자신과 내적)
 $0^T 0$ 로 표현

$$\begin{bmatrix} x_1 \text{ 성능} & x_2 \text{ 가격} & y = \text{노조비판의량} & w^T x = \hat{y} \text{ (추정량)} \\ 1 & 2 & 3 & 2 \\ 2 & 1 & 2 & 0 \\ 3 & 5 & 4 & 2 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

두개는 비교

x_1	y
2	3
1	2
5	4

→ w 를 2개 구해야함. 기변수가 2이니까



$$\begin{array}{cc} \hat{y}_1 & \hat{y}_2 \\ 2 & 0 \\ 0 & 0 \\ 2 & 1 \end{array}$$

$$\textcircled{1} \sum (y_i - \hat{y}_i)^2 = (3-2)^2 + (2-0)^2 + (4-2)^2 = 9$$

$$\textcircled{2} \sum (y_i - \hat{y}_i)^2 = (3-0)^2 + (2-0)^2 + (4-1)^2 = 9 + 4 + 9 = 22.$$

①번이 18가 더 적으므로 좋다.

↓

w 를 알 수 있음

최소 제곱법

$$\begin{aligned} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) &= (\mathbf{y}^T - \mathbf{w}^T \mathbf{X}^T) (\mathbf{y} - \mathbf{X}\mathbf{w}) \\ &= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{y} - \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w} \end{aligned}$$

목적 함수를 최소화하는 지점을 찾기 위해 목적 함수를 미분해서 0이 되는 지점을 찾으면 다음과 같습니다.

$$\frac{\partial (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})}{\partial \mathbf{w}} = 2\mathbf{X}^T \mathbf{X}\mathbf{w} - \mathbf{X}^T \mathbf{y} - \mathbf{X}^T \mathbf{y} \equiv 0$$

W에 대해
미분

$$\Leftrightarrow 2\mathbf{X}^T \mathbf{X}\mathbf{w} = 2\mathbf{X}^T \mathbf{y}$$

$$\Leftrightarrow \hat{\mathbf{w}} = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1}} \mathbf{X}^T \mathbf{y}$$

$\mathbf{X}^T \mathbf{X}$ 의 행렬식이 0이면
역행렬 구할 수 없고
 $\mathbf{w}$ 를 얻을 수 없다.

최소 제곱법 with 제약식

if $w \leq 0.3$ 이어야 한다면?

$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad \text{s.t.} \quad \sum_{i=1}^p w_i^2 \leq t$$

→ 라그랑주 프래멀 함수로
표현해서 미분할 수
있도록 함.

제약

제약식에는 여러 가지 종류가 있는데, 위 제약식은 L2 제약식을 나타냅니다.

위 식을 행렬 형태로 나타내면 다음과 같이 표현할 수 있습니다.

$$\min (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}), \quad \text{s.t.} \quad \|\mathbf{w}\|_2^2 \leq t$$

(ex) 100x2
행렬
2x1
열벡터

$\mathbf{X}\mathbf{w}$ 해야 계산이 됨

AI/BigData 인텐시브 과정

Section 1. 최적화

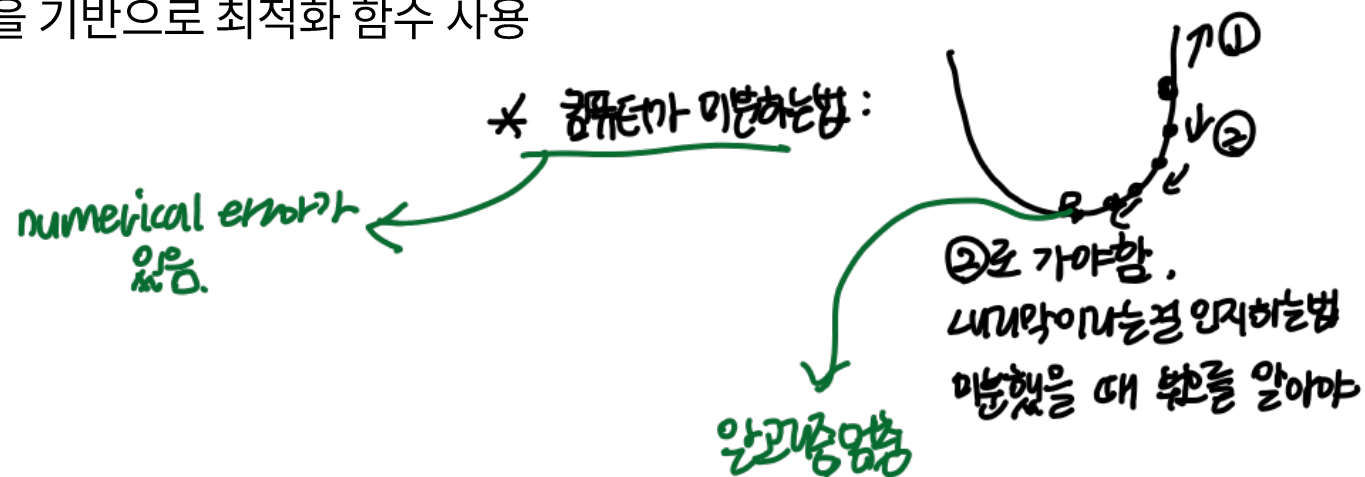
Section 1-8. 그래디언트 디센트

옵티마이저(optimizer)

옵티마이저(optimizer): 최적화 목적 함수인 손실 함수를 기반으로 매 학습 때마다 어떻게 업데이트 할지 결정

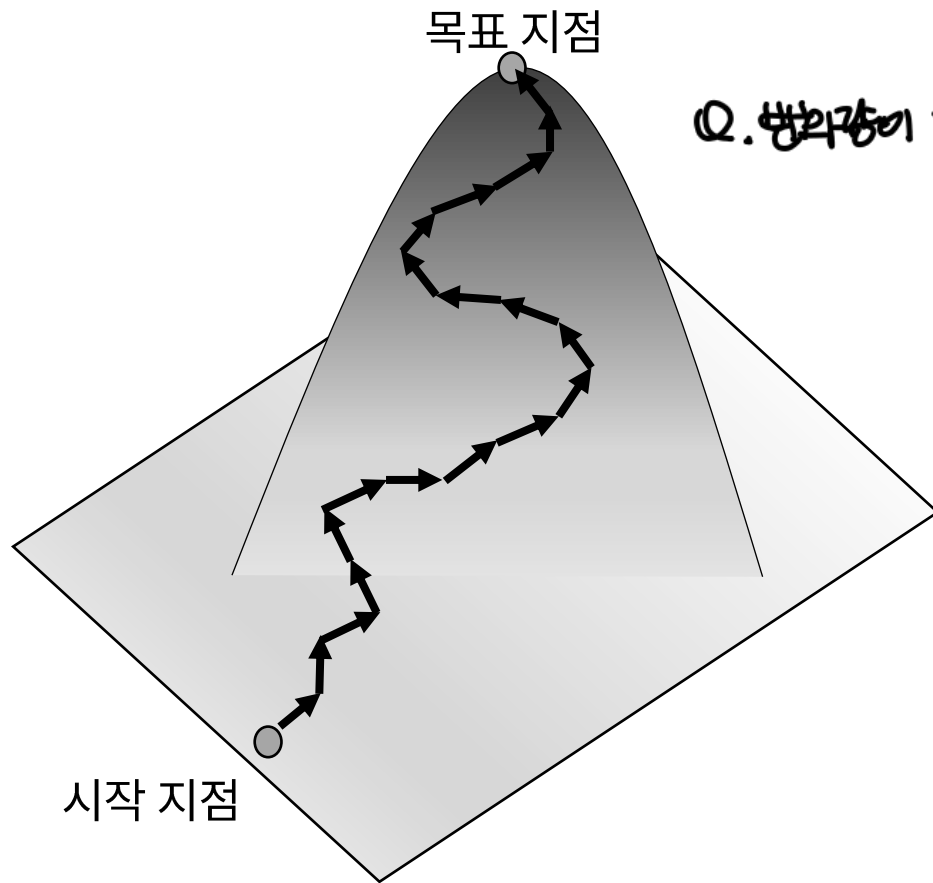
그래디언트 디센트(gradient descent): 우리말로 경사 하강법이라고 불리는 옵티마이저 방법 중 하나.

여러 딥러닝 알고리즘들은 그래디언트 디센트 알고리즘을 기반으로 최적화 함수 사용

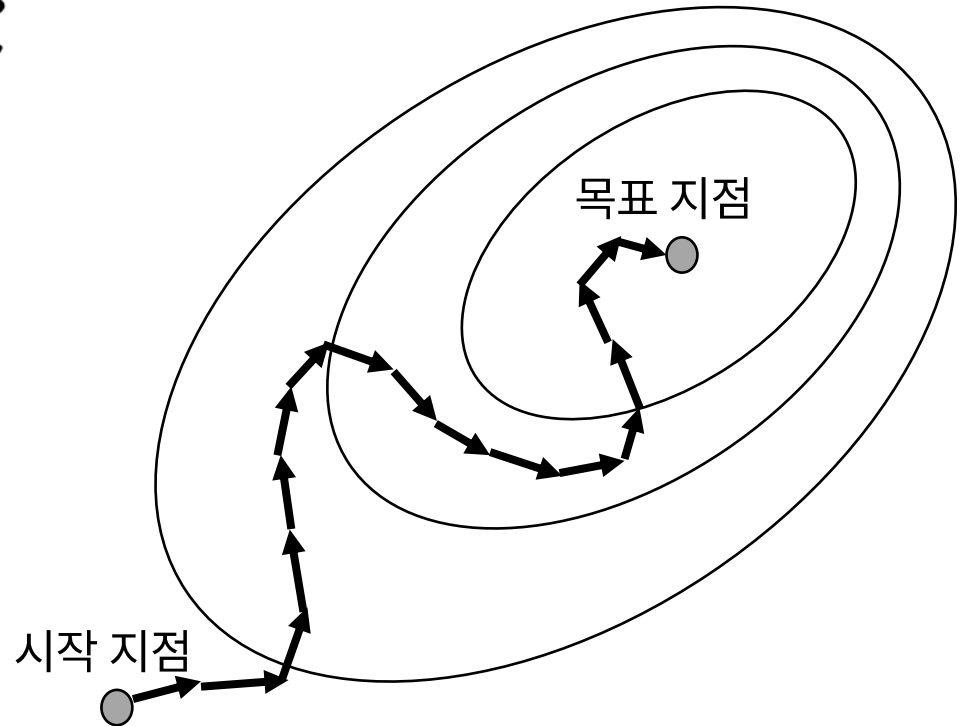


* stop을 어떻게 이룸? → optimizer.

그래디언트 디센트(gradient descent)



Q. 방향이 눈썹으로?



그래디언트 디센트(gradient descent)

그래디언트 디센트: 비용 함수의 미분값을 이용하는 방법

미분: 기울기 혹은 순간 변화율을 의미

앞서 1차 미분 조건에서 언급했듯, 미분해서 기울기가 0이 되는 지점은 최적값의 조건이 됨.

미분해서 0이 되는 지점을 찾기 위해 그래디언트 디센트는 최초로 구한 미분값의 반대 방향으로 이동시킨 후 이동한 지점에서 다시 미분함

이러한 과정을 반복해서 최적값에 가까워짐.

그라디언트 디센트(gradient descent)

θ : 파라미터

$J(\theta)$: 목적 함수

그라디언트 디센트

- (1) 초기 파라미터 θ_1 에 해당하는 $J(\theta_1)$ 의 미분값을 구합니다.
- (2) 1에서 구한 미분값의 반대 방향으로 θ_1 을 $\nabla J(\theta_1)$ 만큼 이동시킵니다. 이동 후 지점을 θ_2 라고 합니다.
- (3) 1, 2단계를 반복합니다.

위 그라디언트 디센트 과정 2단계를 수식으로 나타내면 아래와 같습니다.

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

$$\Delta\theta_t = -\eta \nabla J(\theta_t)$$

그라디언트 디센트(gradient descent)

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

$$\Delta\theta_t = -\eta \nabla J(\theta_t)$$

에타
(eta)

gradient, Jacobian의 J.

파라미터 θ_t 는 t 단계에서의 θ 를 의미

이제: 다음스텝의 위치.

$\nabla J(\theta_t)$ 는 목적 함수 $J(\theta_t)$ 의 미분값 즉, 변화율을 의미

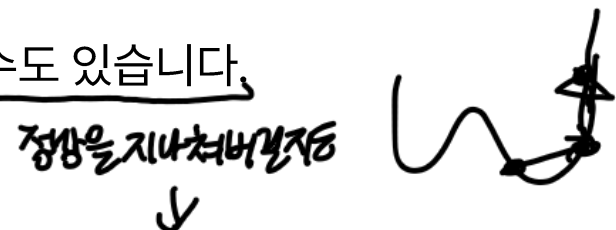
η 는 학습률을 의미 (learning rate)

학습률(learning rate)

η 을 통해서 한 스텝의 크기를 정하는 것. $\rightarrow$ 우리가 정하는 것.
구하는 공식은 없음.

학습률(learning rate)이란 미분값의 반대 방향으로 움직이는 데 얼마나 움직일지 정합니다.

만약 학습률을 너무 높게 설정하면 한 번에 이동하는 값이 너무 커져서 최적값을 구하지 못할 수도 있습니다.



반대로 학습률을 너무 낮게 설정하면 최적값을 구하는 데 많은 시간이 소요됩니다.

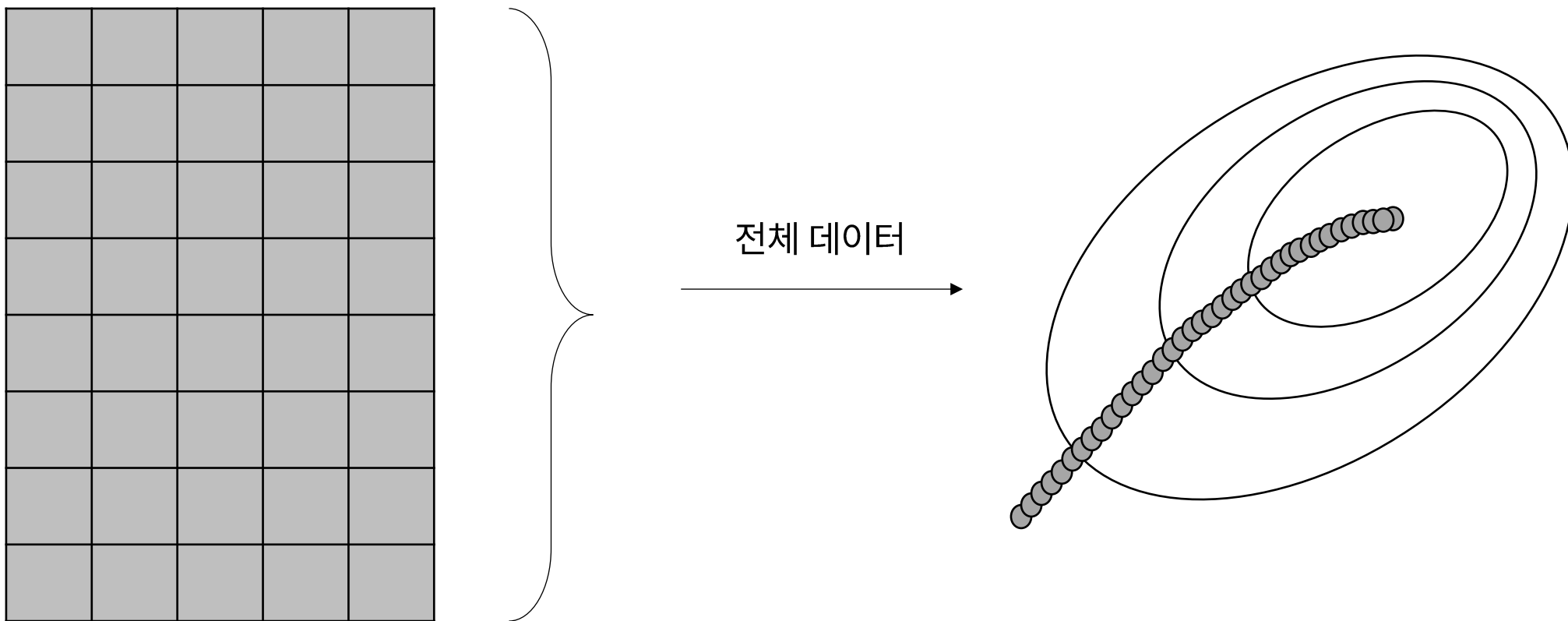
* 중간에 더 적절한 learning rate를 찾아야.

$\hookrightarrow$ 싱크로....

loss가 폭발하는 일 생김
(gradient explosion)

그래디언트 디센트(gradient descent)

그래디언트 디센트는 전체 데이터를 이용해 파라미터가 최적값으로 수렴하게끔 학습시킵니다.



마이너스(-)를 붙이는 이유

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

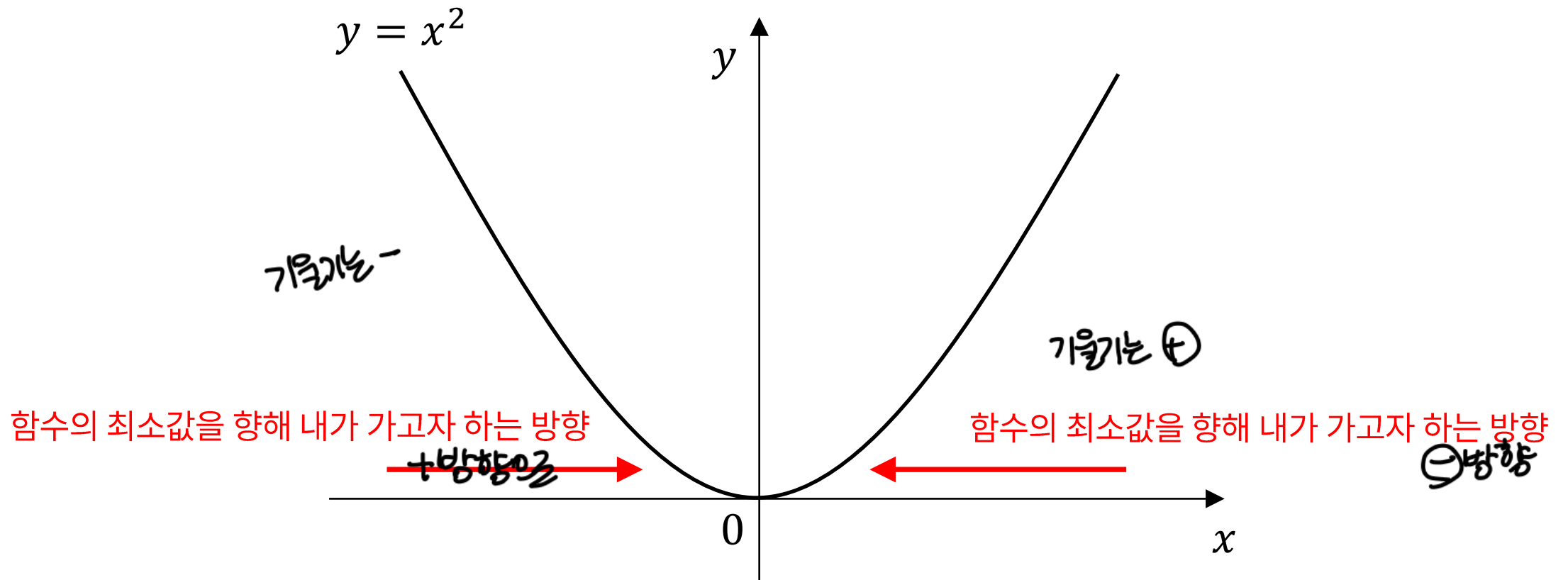
$$\Delta\theta_t = -\eta \nabla J(\theta_t)$$

마이너스 왜 붙이나요?

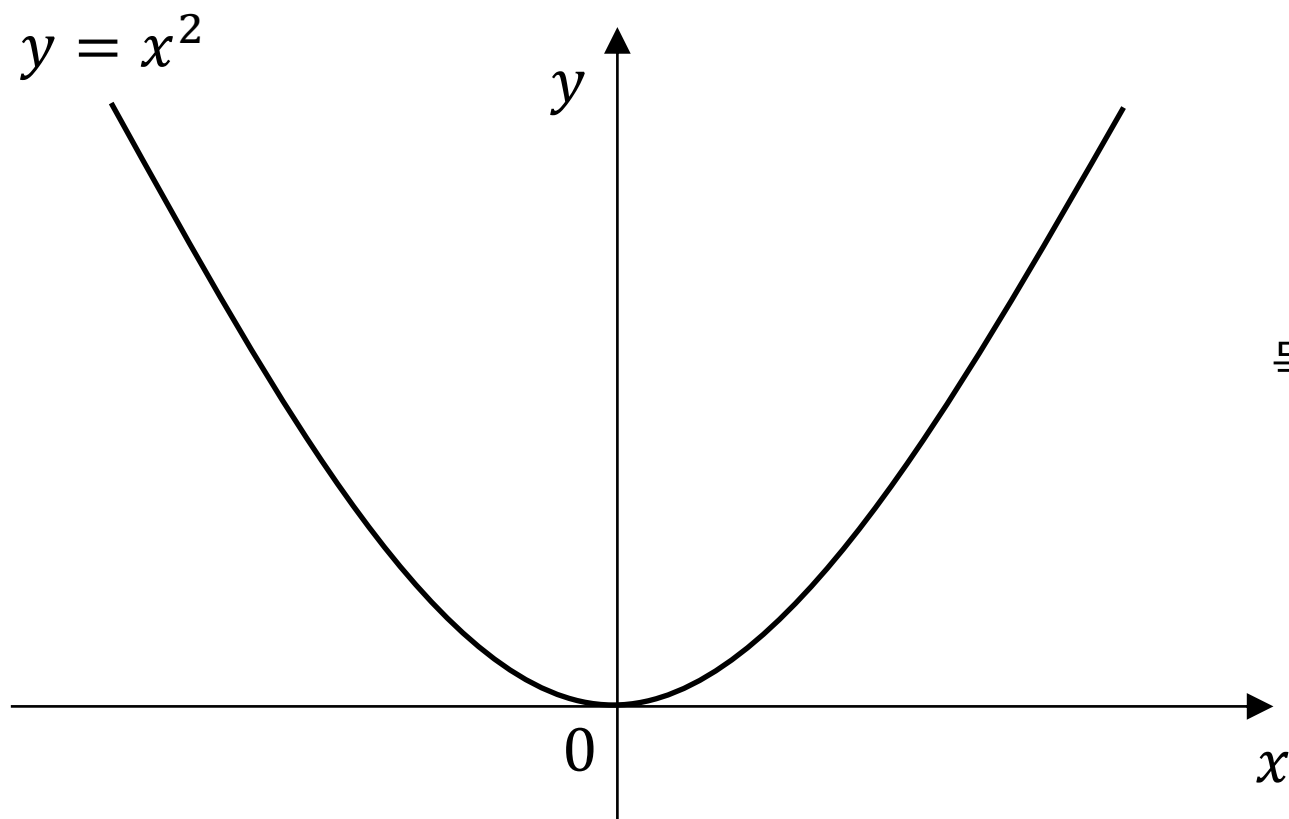
반대로 가기 위해서 입니다

반대로요?!

마이너스(-)를 붙이는 이유



마이너스(-)를 붙이는 이유



목적 함수: $y = x^2$

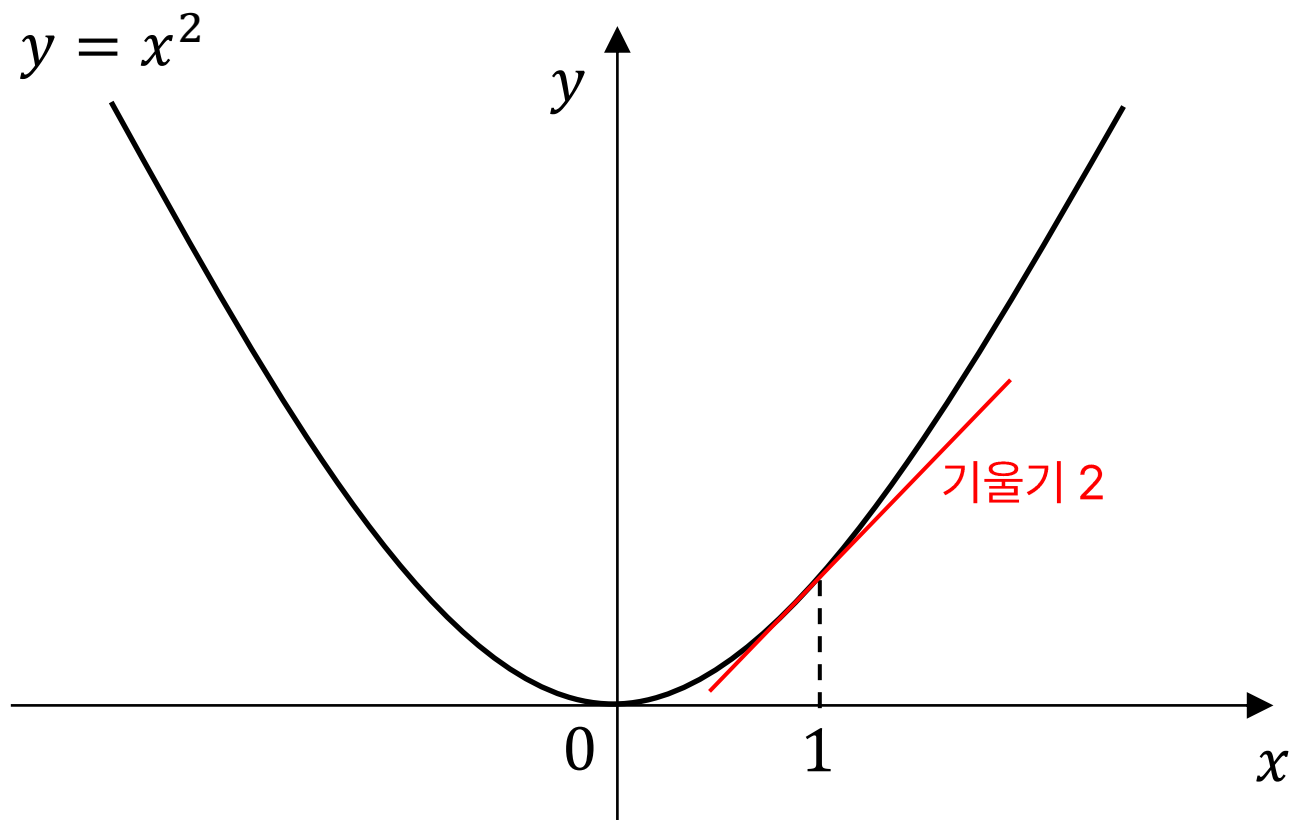
목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$



x값에 따른 기울기를 의미

x>0 이면 기울기는 양수,
x<0 이면 기울기는 음수

마이너스(-)를 붙이는 이유



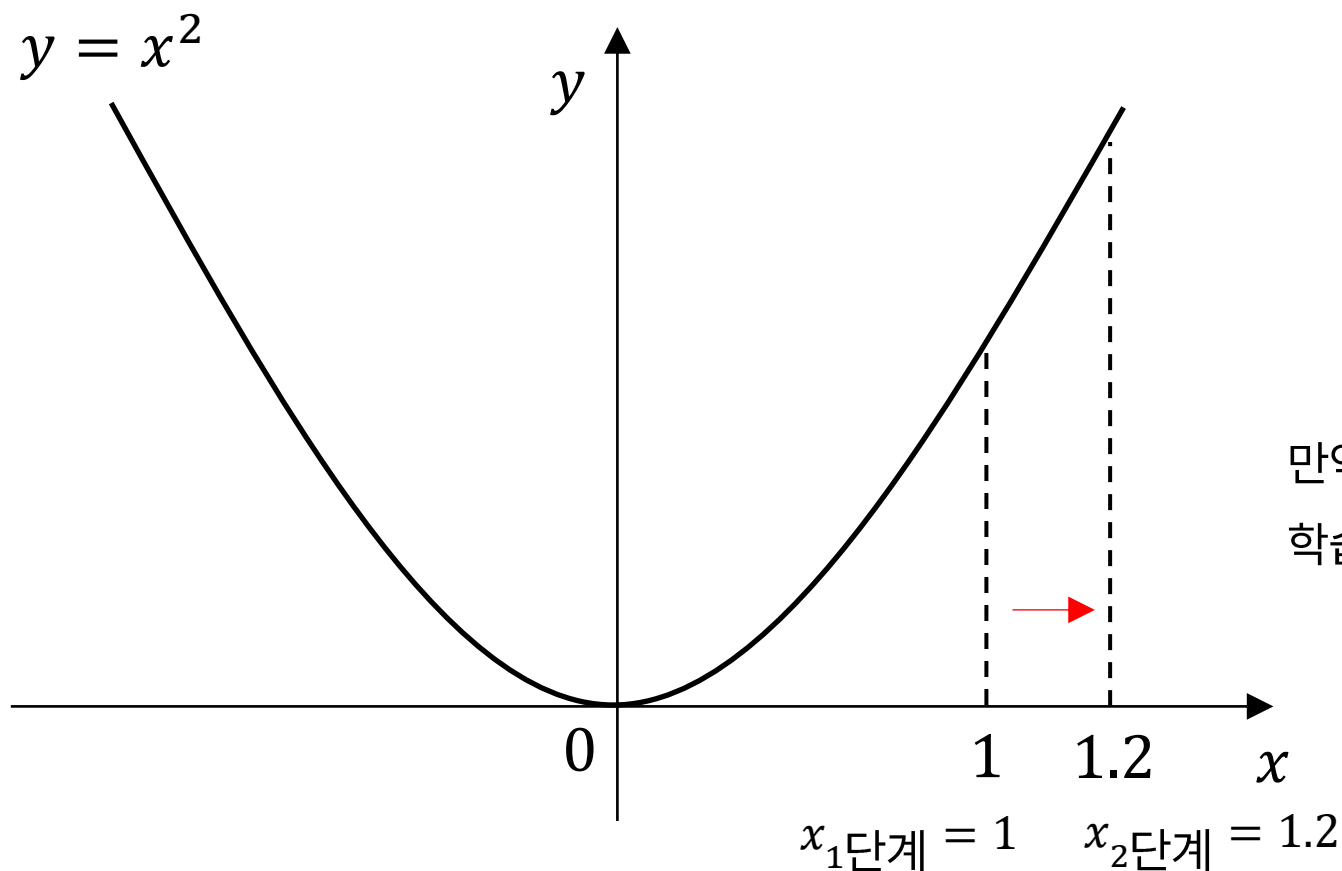
목적 함수: $y = x^2$

목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$

예를 들어

$$x = 1 \quad \text{이면 } \frac{dy}{dx} = 2x = 2 \times 1 = 2$$

마이너스(-)를 붙이는 이유



목적 함수: $y = x^2$

목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$

예를 들어

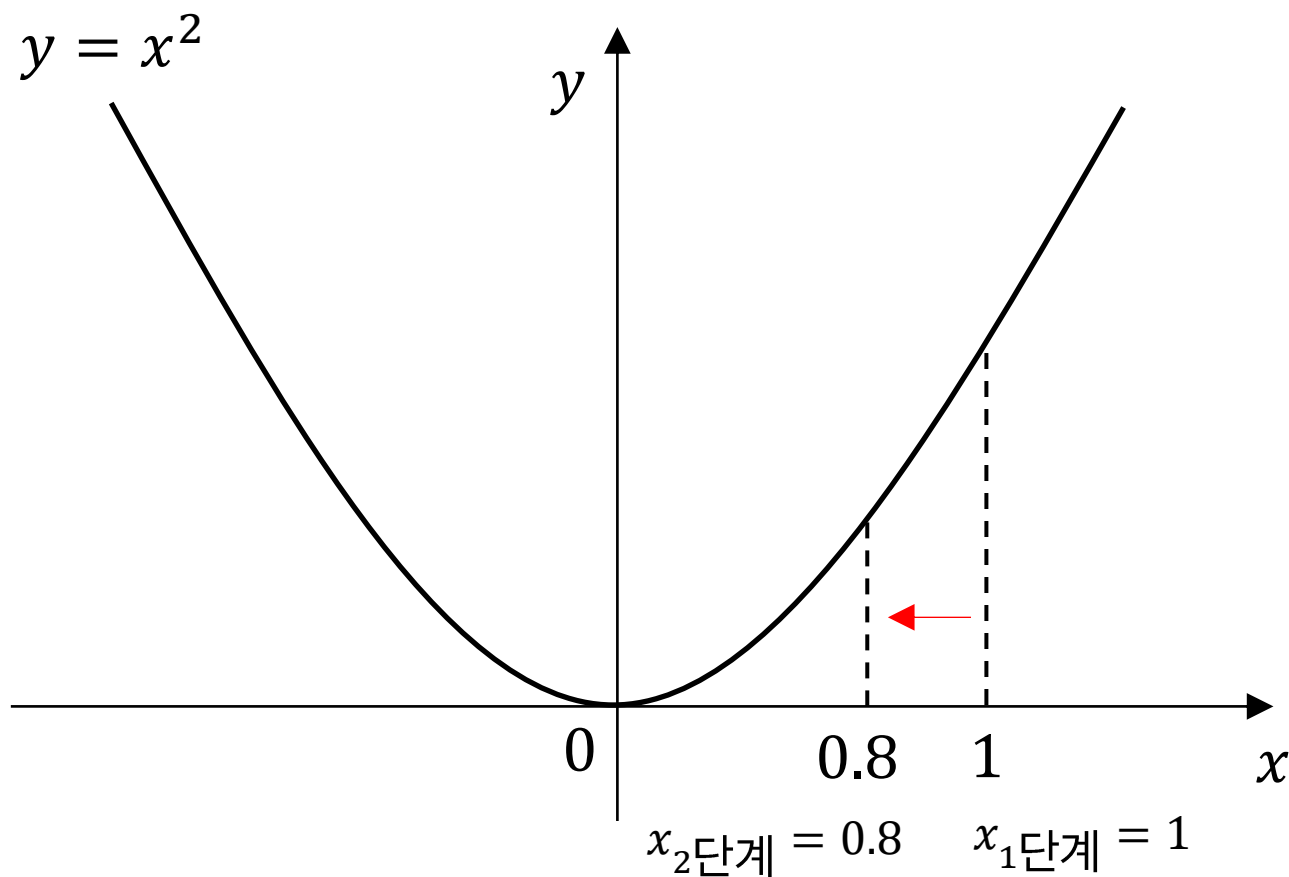
$$x = 1 \quad \text{이면 } \frac{dy}{dx} = 2x = 2 \times 1 = 2$$

만약 이 상태에서 그래디언트 만큼 양수(+) 방향으로 이동한다?
학습률이 0.1이라고 하면

$$x_1\text{단계} = 1 \quad x_2\text{단계} = 1 + 2 \cdot 0.1 = 1.2$$

내가 가야하는 방향과 멀어짐
그래서 반대로 가야합니다!

마이너스(-)를 붙이는 이유



목적 함수: $y = x^2$

목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$

예를 들어

$$x = 1 \quad \text{이면 } \frac{dy}{dx} = 2x = 2 \times 1 = 2$$

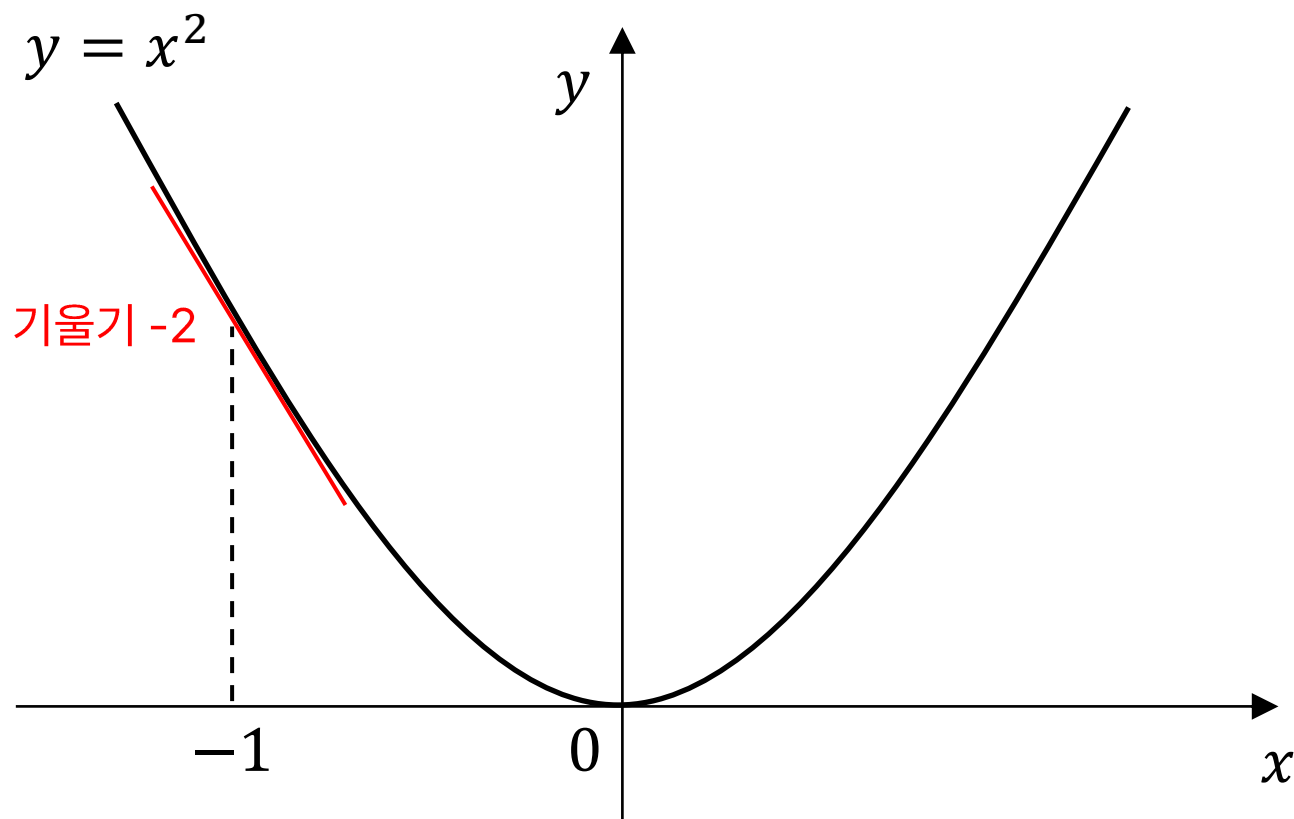
반대로 가면? 학습률이 0.1이라고 하면

$$x_1\text{단계} = 1 \quad x_2\text{단계} = 1 - 2 \cdot 0.1 = 0.8$$

반대 방향으로 가면 내가 가고자 하는 방향으로 가는 것을 볼 수 있음

2번째 단계의 반대방향을 가야함

마이너스(-)를 붙이는 이유



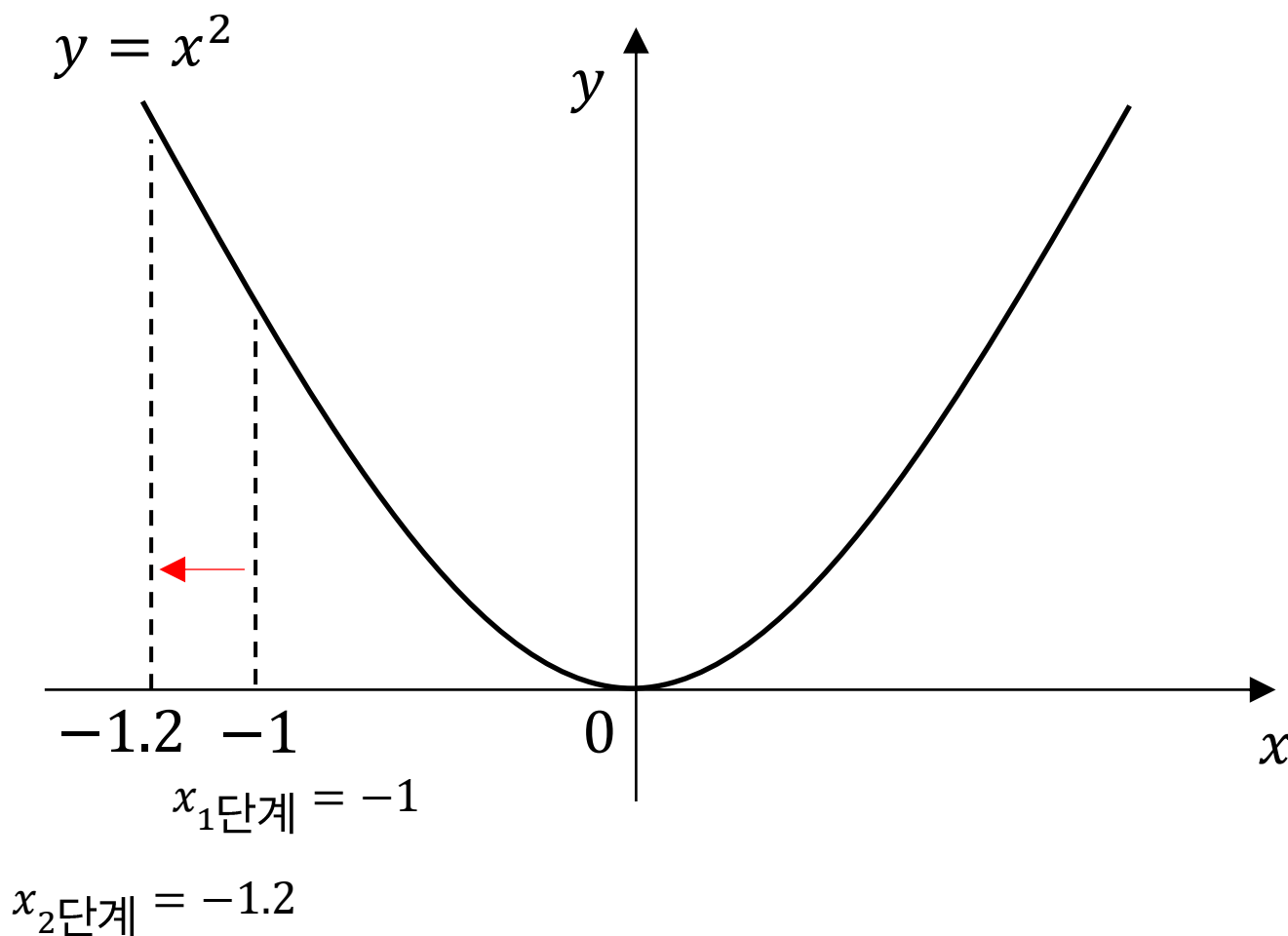
목적 함수: $y = x^2$

목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$

예를 들어

$$x = -1 \text{ 이면 } \frac{dy}{dx} = 2x = 2 \times (-1) = -2$$

마이너스(-)를 붙이는 이유



목적 함수: $y = x^2$

목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$

예를 들어

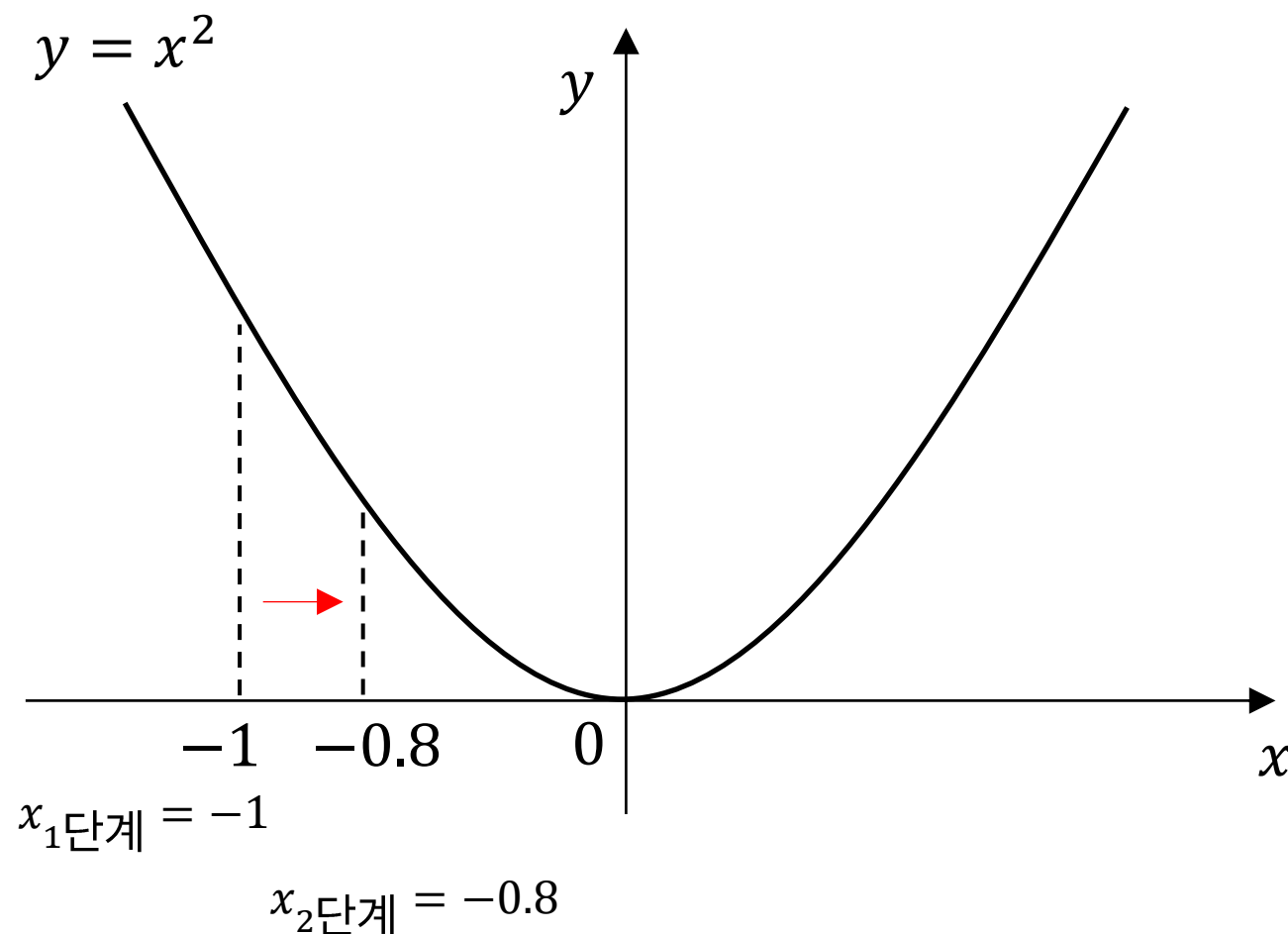
$$x = -1 \text{ 이면 } \frac{dy}{dx} = 2x = 2 \times (-1) = -2$$

만약 이 상태에서 그래디언트 만큼 양수(+) 방향으로 이동한다?
학습률이 0.1이라고 하면

$$x_1\text{단계} = -1 \quad x_2\text{단계} = -1 + (-2 \cdot 0.1) = -1.2$$

내가 가야하는 방향과 멀어짐
그래서 반대로 가야합니다!

마이너스(-)를 붙이는 이유



목적 함수: $y = x^2$

목적 함수의 그래디언트(미분값): $\frac{dy}{dx} = 2x$

예를 들어

$$x = -1 \text{ 이면 } \frac{dy}{dx} = 2x = 2 \times (-1) = -2$$

반대로 가면? 학습률이 0.1이라고 하면

$$x_1\text{단계} = -1 \quad x_2\text{단계} = -1 - (-2 \cdot 0.1) = -0.8$$

반대 방향으로 가면 내가 가고자 하는 방향으로 가는 것을 볼 수 있음

SGD

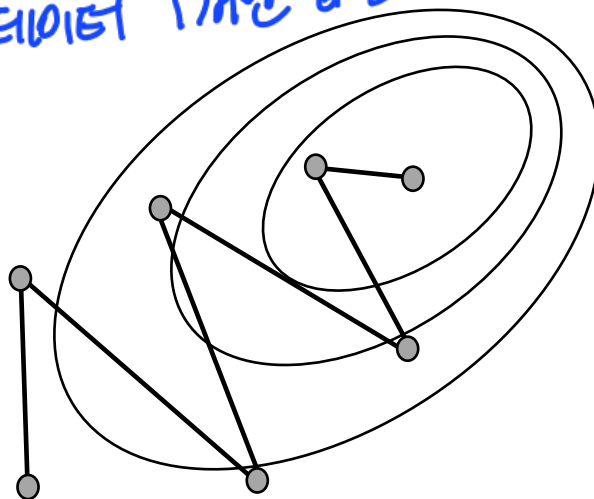
확률적 그라디언트 디센트 (stochastic gradient descent)

이 방법은 모형 학습을 빠르게 시키기 위해 고안된 방법으로 전체 데이터에 대해 수행하는 것이 아닌 전체 데이터에서 추출한 샘플 트레이닝 데이터를 사용해 최적값을 찾습니다
(연산량 줄여줌)

쉽게 말해, 확률적 그라디언트 디센트는 그라디언트 디센트의 샘플링 버전

샘플 데이터

이제는 데이터 1개만 뽑는거 (?) → 불완전해서 대배치



$$\theta_{t+1} = \theta_t + \Delta \theta_t$$

$$\Delta \theta_t = -\eta \nabla J(\theta_t; \mathbf{x}^{(i)}, y^{(i)})$$

확률적 그라디언트 디센트(stochastic gradient descent)

장점

SGD는 전체 트레이닝 데이터가 아닌 일부를 이용하므로 배치 그라디언트 디센트보다 속도가 빠름

시스템 리소스도 적게 요구합니다.

단점

SGD는 수렴 속도가 빠른 만큼 잘못하면 최적해를 벗어나는 경우도 있습니다

이를 방지하기 위해 적절한 학습률을 정하는 것이 중요한데 학습률을 정하기 어렵다는 단점이 존재

만약 SGD 과정에서 최적해를 벗어나는 경우, 학습률을 낮추면 최적해에 수렴하는 것을 볼 수 있습니다.

미니배치 그레디언트 디센트

→ 외증에는 이것을 SGD라고 부름

미니 배치: 사이즈가 작은 배치 데이터

일부분을 골라선자,

$\mathbf{x}^{(i:i+n)}$ 는 i 번째 데이터 포인트부터 $i + n$ 번째 데이터 포인트를 포함한다는 의미

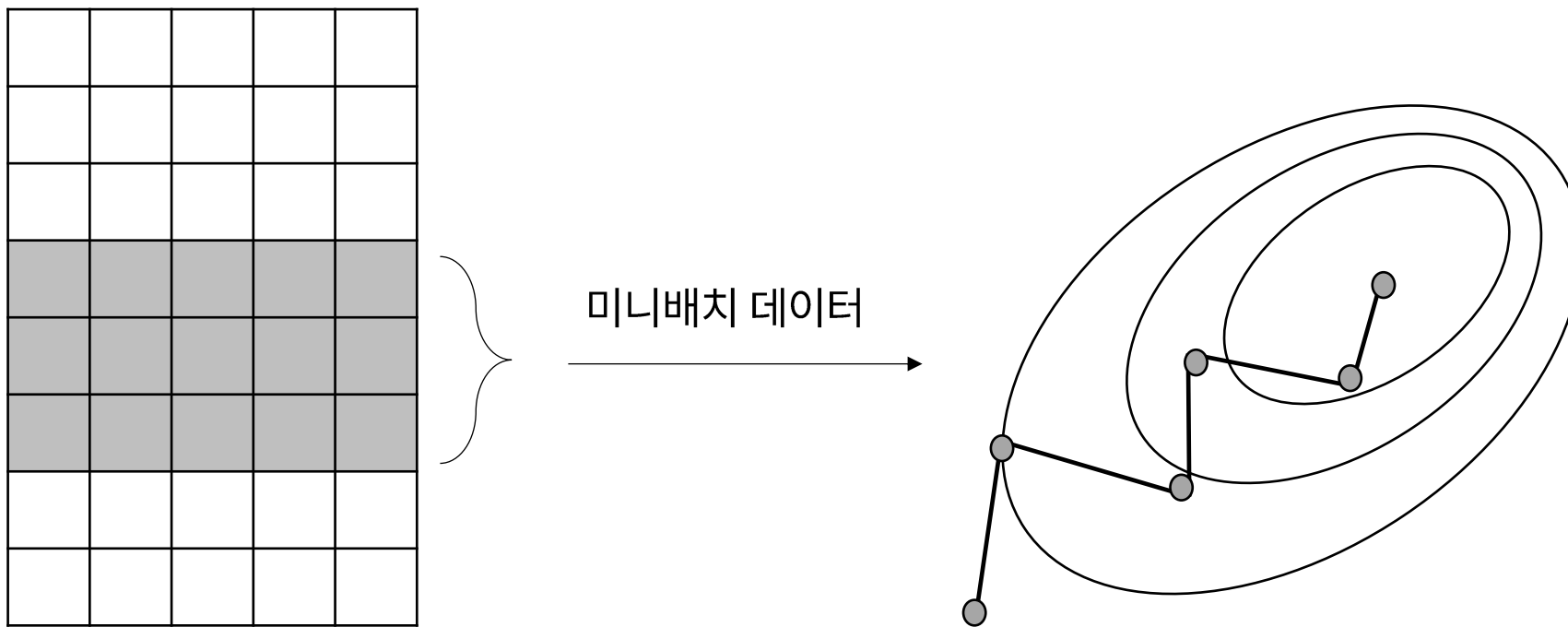
i 번째 데이터 포인트

⋮

$i + n$ 번째 데이터 포인트

미니배치 그래디언트 디센트

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \Delta \boldsymbol{\theta}_t$$
$$\Delta \boldsymbol{\theta}_t = -\eta \nabla J(\boldsymbol{\theta}_t; \mathbf{x}^{(i:i+n)}, \mathbf{y}^{(i:n)})$$

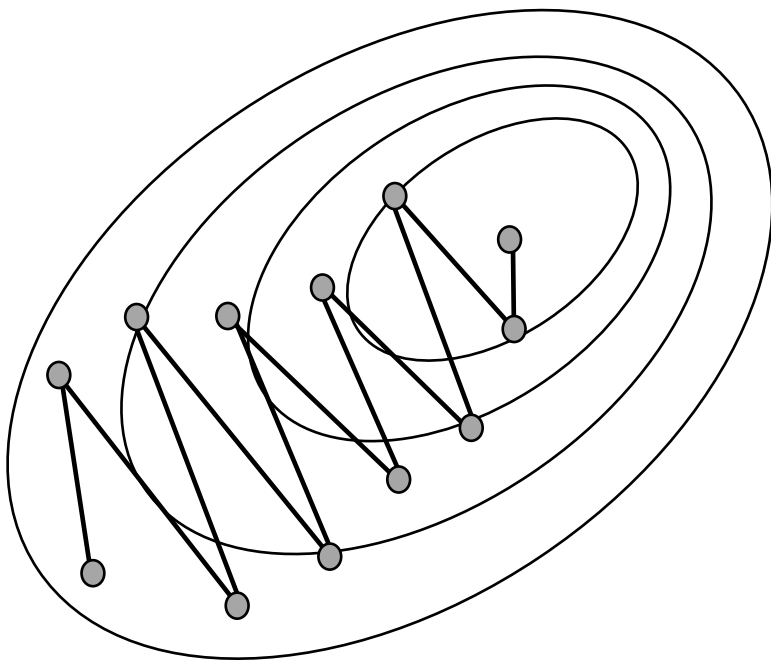


모멘텀(momentum)

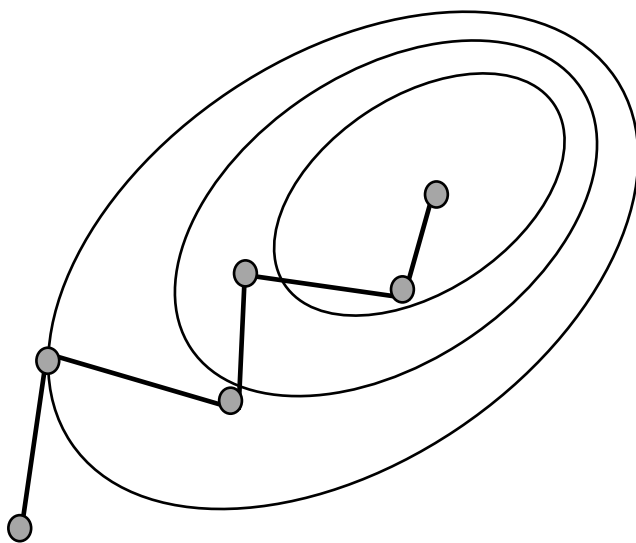
학습 속도를 가속화하는 방법으로 모멘텀(momentum)이라는 방법이 등장

하이퍼파라미터 γ 는 모멘텀을 얼마나 줄지 결정하는 값으로 일반적으로 0.9 근방의 값으로 설정

모멘텀 미적용



모멘텀 적용



$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla J(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t$$

감사합니다.

Q & A